

Lecture 4: Transformer Language Modeling



NYU

Greg Durrett

Administrative details and recap

Neural Language Modeling

Attention

Self-attention

Transformers

Positional Encodings

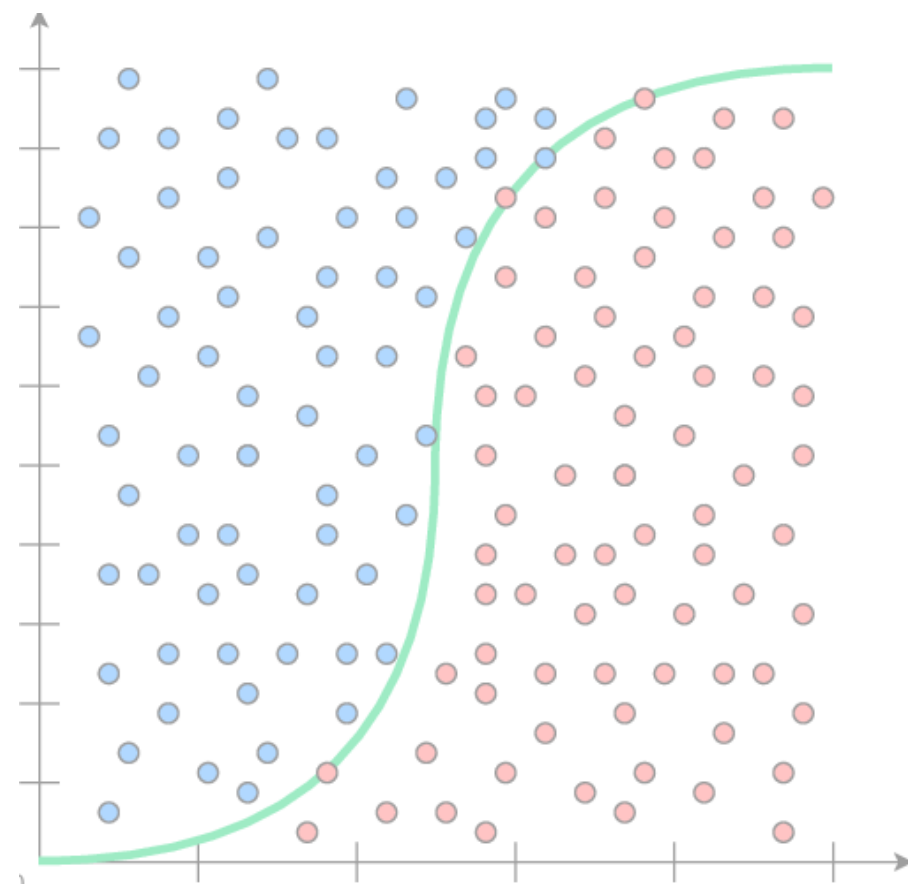
Transformer Language Modeling

Recap

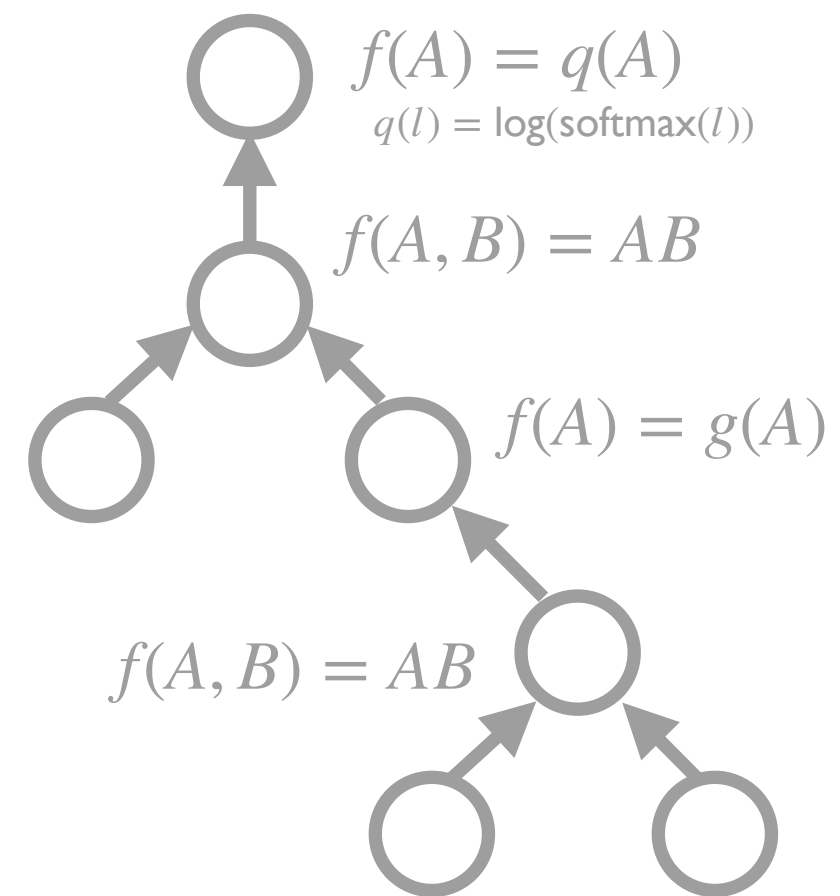
Announcements

- ▶ A2 due in two weeks, Quiz 2 in two weeks
- ▶ A1, Quiz 1 returned soon

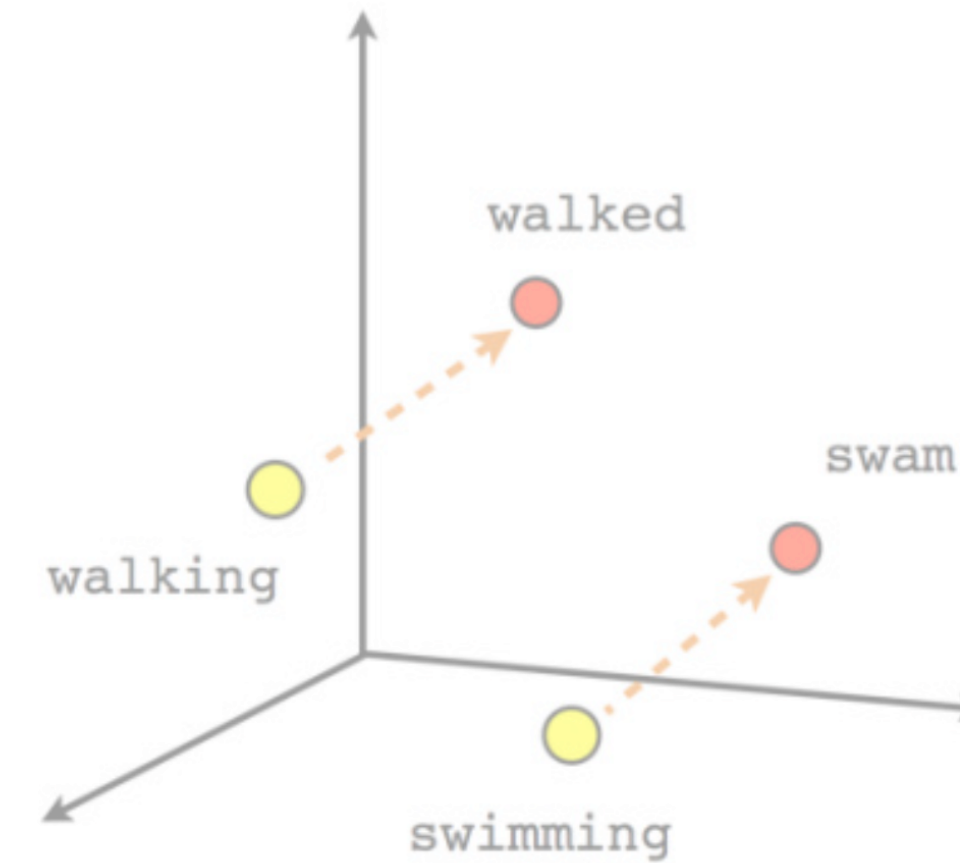
Overview



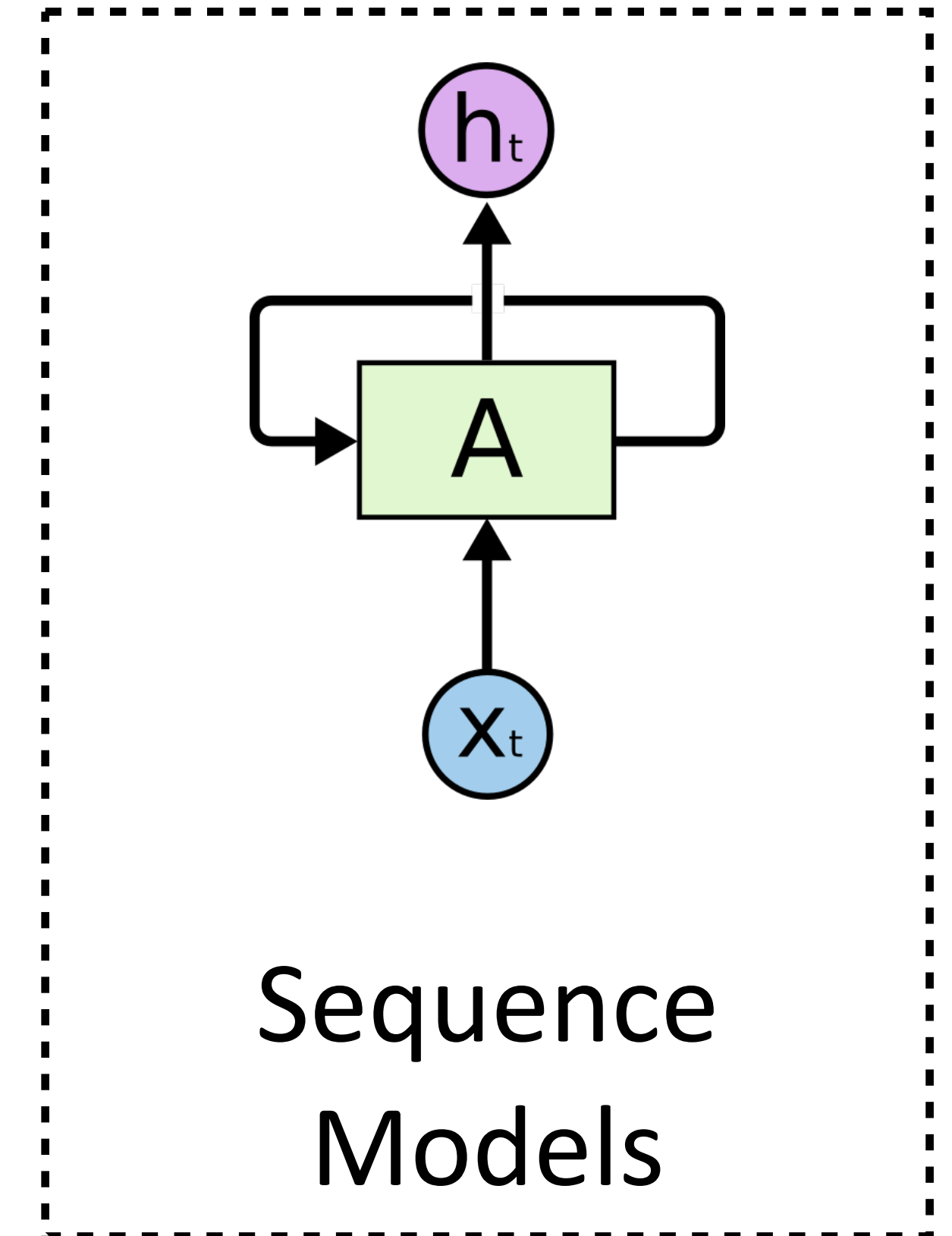
Logistic
Regression



Neural
Networks



Word
Embeddings



Sequence
Models

Recap: Sequence Modeling

- ▶ Model that can keep a “state” to capture interaction among the words
- ▶ *How can you not see this movie?* vs. *You should not see this movie.*
- ▶ Example: part of speech tagging relies on contextual information

PRP VBP DT NN
I record the song

What’s the potential ambiguity here? *record* can also be *NN*

Recap: Language Modeling

- ▶ Goal: Assign probabilities $P(x)$ to strings of text x
Input: many observations of training strings
Output: system capable of computing $P(x)$

- ▶ Decompose the probability:

$$P(\mathbf{x}) = P(x_1)P(x_2 | x_1)P(x_3 | x_2, x_1) \dots$$

$$\begin{aligned} P(\text{the cat sat on the mat}) &= P(\text{the}) * P(\text{cat}|\text{the}) * P(\text{sat}|\text{the cat}) \\ &\quad * P(\text{on}|\text{the cat sat}) * P(\text{the}|\text{the cat sat on}) \\ &\quad * P(\text{mat}|\text{the cat sat on the}) \end{aligned}$$

- ▶ Fundamental task in both linguistics and NLP: can we determine if a sentence is *acceptable* or not?

Recap: RNNs

- ▶ Maps from dense input sequence to dense hidden state representation sequence

$$\mathbf{x}_1, \dots, \mathbf{x}_n \rightarrow h_1, \dots, h_n$$

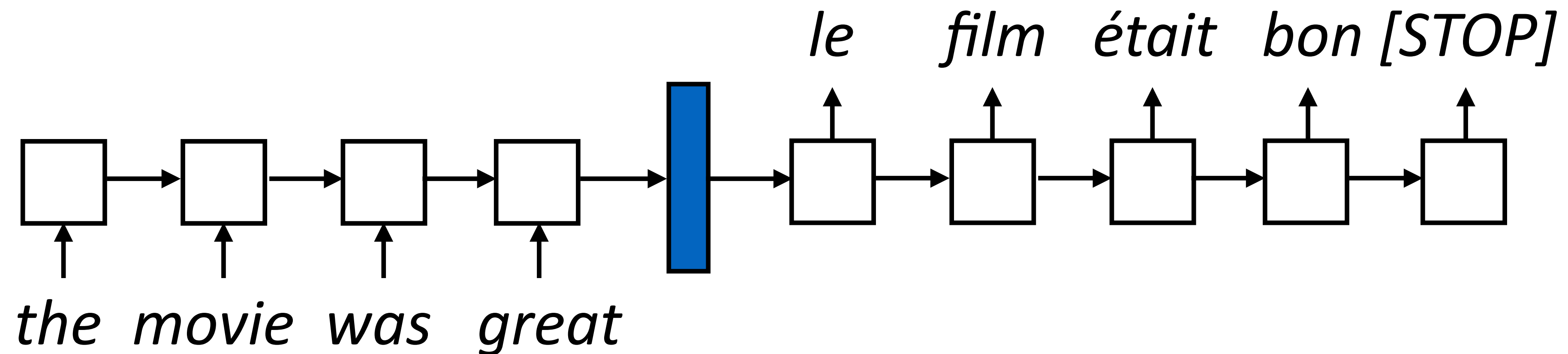
- $S = \mathbb{R}^{d_{hid}}$ - hidden state space ($h_1, h_2 \dots$)
- $\Sigma = \mathbb{R}^{d_{in}}$ - input state space ($x_1, x_2 \dots$)
- $s_0 \in S$ - initial state vector (h_0)
- $R : \mathbb{R}^{d_{in}} \times \mathbb{R}^{d_{hid}} \rightarrow \mathbb{R}^{d_{hid}}$ - transition function

- ▶ For all $i \in \{1, \dots, n\}$, $h_i = R(h_{i-1}, \mathbf{x}_i)$
g: nonlinearity (e.g. tanh)
- ▶ Simple definition of R: $R(h_{i-1}, x_i) = g(Wh_{i-1} + Ux_i + b)$
- ▶ R is parameterized, where the parameters are shared across all steps.

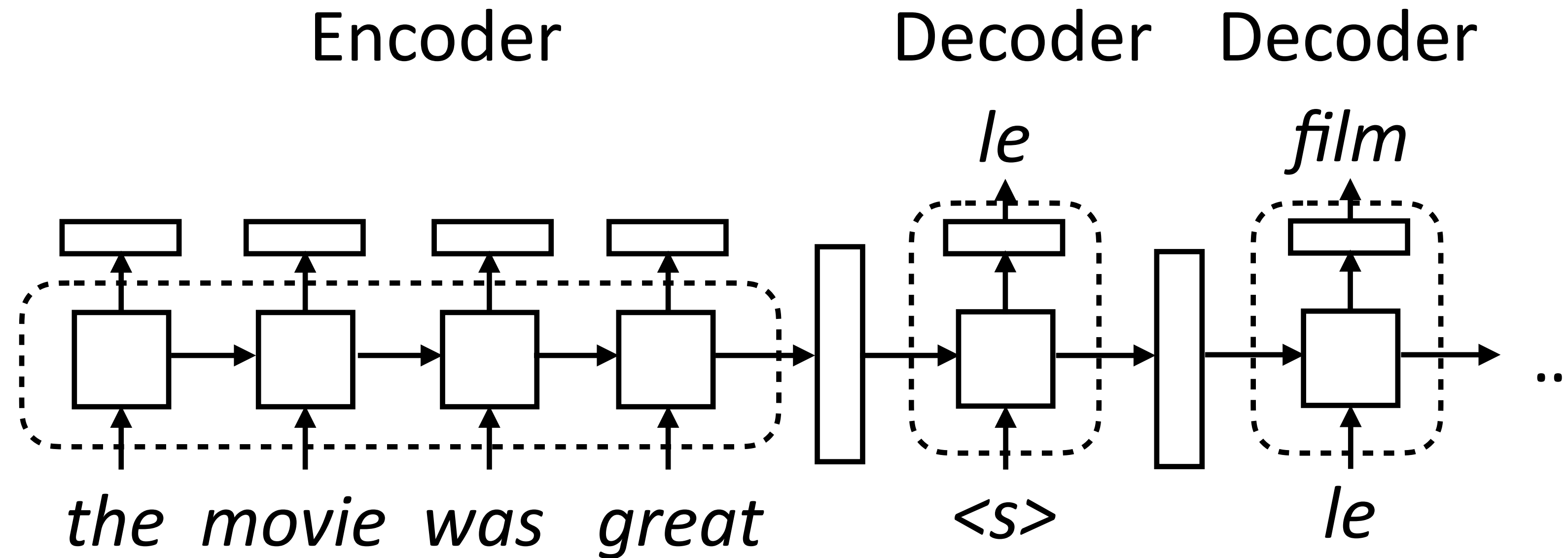
$$h_4 = R(h_3, \mathbf{x}_4) = \dots = R(R(R(R(h_0, \mathbf{x}_1), \mathbf{x}_2), \mathbf{x}_3), \mathbf{x}_4)$$

Recap: Seq2Seq Modeling with RNNs

- ▶ Input: a sequence of tokens
- ▶ Output: a sequence of tokens (of *arbitrary* length)



Seq2Seq Modeling with RNNs



- ▶ Encoder: a RNN encoding a sequence of tokens, produces a vector.
- ▶ Decoder: separate RNN module (**different** parameters).
 - ▶ Takes two inputs: hidden state and previous token.
 - ▶ Outputs token and a new hidden state.

Administrative details and recap

Neural Language Modeling

Attention

Self-attention

Transformers

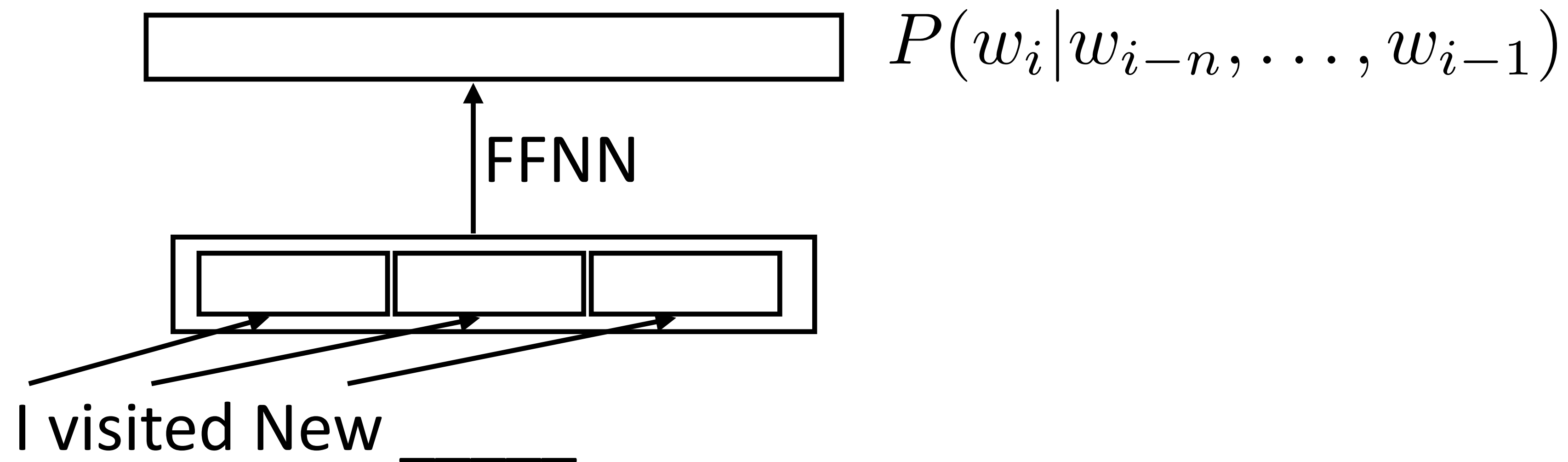
Positional Encodings

Transformer Language Modeling

Neural Language Modeling

Neural Language Models

- ▶ Early work: feedforward neural networks looking at context



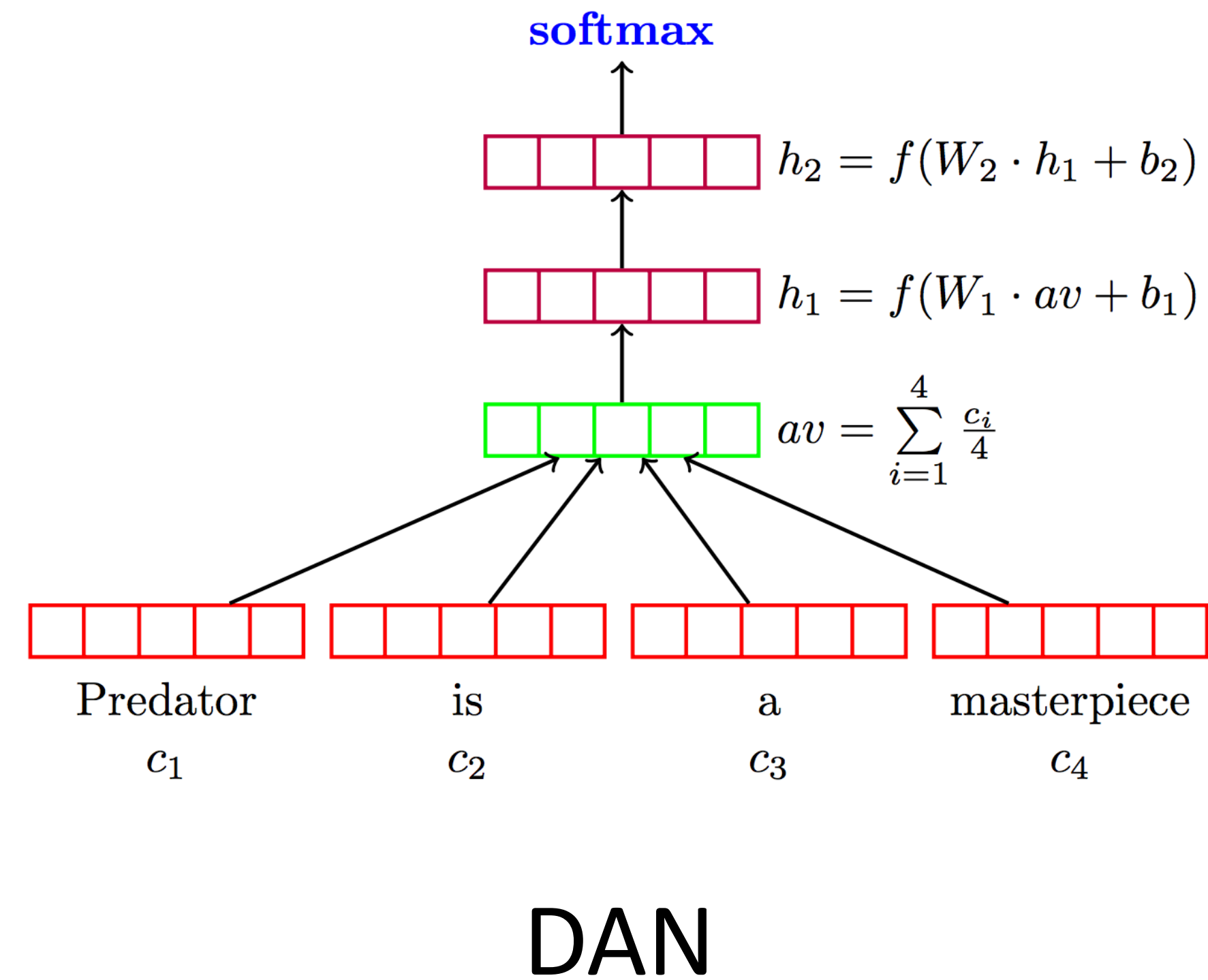
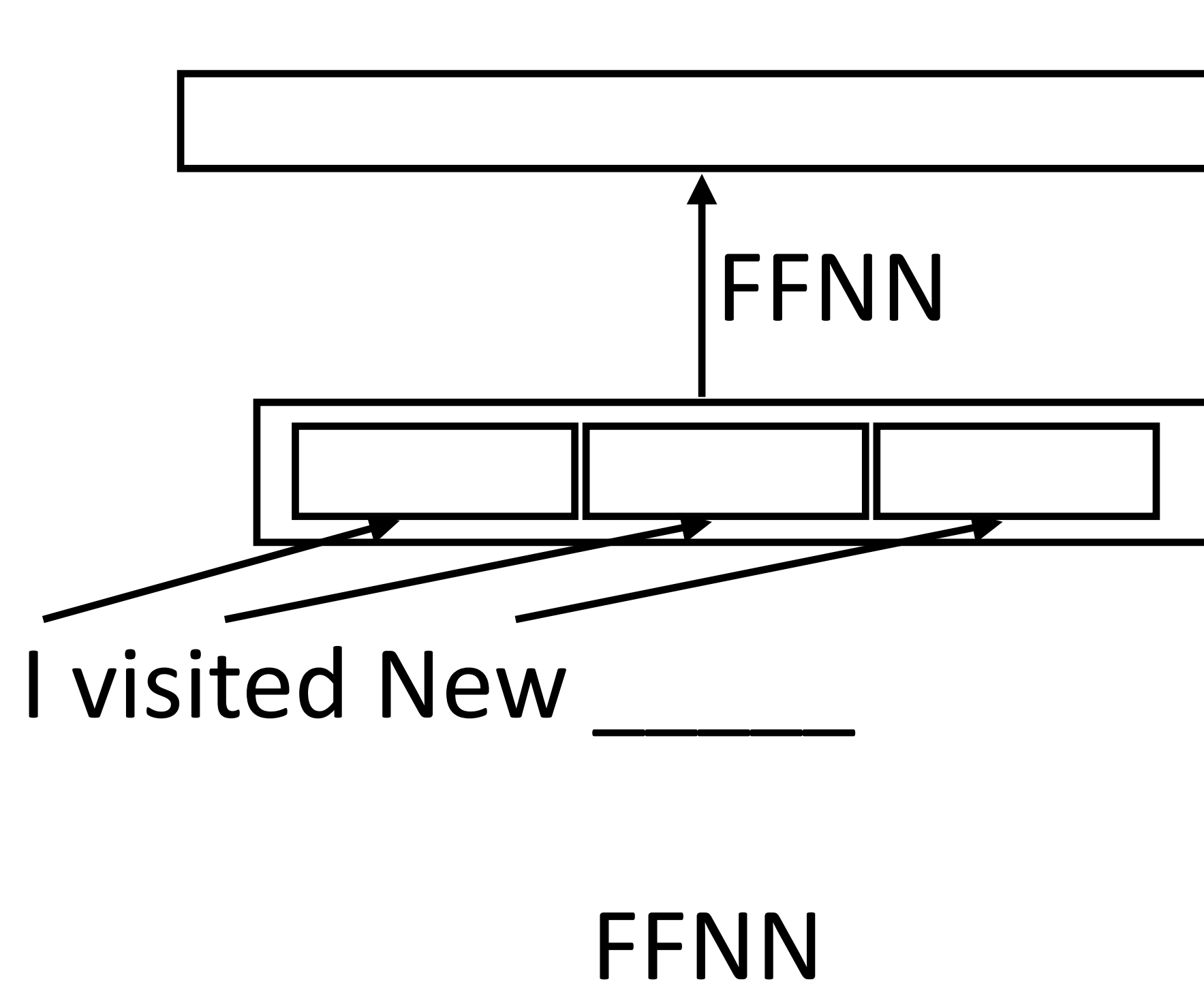
- ▶ Slow to train over lots of data! But otherwise this seems okay?

Problems with FFNNs

x = I visited New York. I had a really fun time going up the ____

- ▶ What are some words that can show up here? How do we know?
- ▶ What do we learn from this example?

Challenges of Neural Language Modeling



- Advantages and disadvantages of these?

Contextualized Embeddings

- ▶ RNNs and Transformers (and other models) can produce *contextualized embeddings*

$$\mathbf{e} = (e_1, e_2, \dots, e_n) \quad e_i = f(x_1, x_2, \dots, x_i)$$

$$\mathbf{x} = (x_1, x_2, \dots, x_n)$$

- ▶ unidirectional representation (only looks at past words)

$\mathbf{x} =$ *I visited New York. I had a really fun time going up the ____*

- ▶ “*the*” stores information about the entire past sequence (theoretically): we can predict the next token in a simple way
- ▶ RNNs give us the e_i via their hidden states...but we’re going to see how Transformers do this!

Administrative details and recap

Neural Language Modeling

Attention

Self-attention

Transformers

Positional Encodings

Transformer Language Modeling

Attention

Running Example

- ▶ Fixed-length sequence of As and Bs

AAAAAA**A**

- ▶ All As = last letter is A; any B = last letter is B

ABAAAA**B**

ABAABA**B**

- ▶ **Attention:** method to access arbitrarily*
far back in context from this point

AAAABA**B**

BAAAAAAAAAAAAAAAAAAAAAAAAAA**B**

- ▶ RNNs generally struggle with this; remembering context for many positions is hard (though of course they can do this simplified example — you can even hand-write weights to do it!)

Keys and Query

- Keys: embedded versions of the sentence; query: what we want to find

Assume $A = [1, 0]$; $B = [0, 1]$ (one-hot encodings of the tokens); call these e_i

Step 1: Compute scores for each key

keys k_i

$[1, 0]$ $[1, 0]$ $[0, 1]$ $[1, 0]$

A A B A

$$s_i = k_i^T q$$

0 0 1 0

query: $q = [0, 1]$ (we want to find Bs)

Attention

Step 1: Compute scores for each key

keys k_i

$[1, 0]$ $[1, 0]$ $[0, 1]$ $[1, 0]$

A A B A

query: $q = [0, 1]$ (we want to find Bs)

$$s_i = k_i^T q$$

0 0 1 0

Step 2: softmax the scores to get probabilities α

0 0 1 0 $\Rightarrow (1/6, 1/6, 1/2, 1/6)$ if we assume $e=3$

Step 3: compute output values by multiplying embs. by alpha + summing

$$\text{result} = \sum(\alpha_i e_i) = 1/6 [1, 0] + 1/6 [1, 0] + 1/2 [0, 1] + 1/6 [1, 0] = [1/2, 1/2]$$

Attention

keys k_i

$[1, 0]$ $[1, 0]$ $[0, 1]$ $[1, 0]$

A A B A

query: $q = [0, 1]$ (we want to find Bs)

$(1/6, 1/6, 1/2, 1/6)$ if we assume $e=3$

$$\text{result} = \sum(\alpha_i e_i) = 1/6 [1, 0] + 1/6 [1, 0] + 1/2 [0, 1] + 1/6 [1, 0] = [1/2, 1/2]$$

How does this differ from just averaging the vectors?

What if we have a very very long sequence?

New Keys

keys k_i

$[1, 0]$ $[1, 0]$ $[0, 1]$ $[1, 0]$

A A B A

query: $q = [0, 1]$ (we want to find Bs)

We can make attention more peaked by not setting keys equal to embeddings.

$$k_i = W^K e_i \quad W^K = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

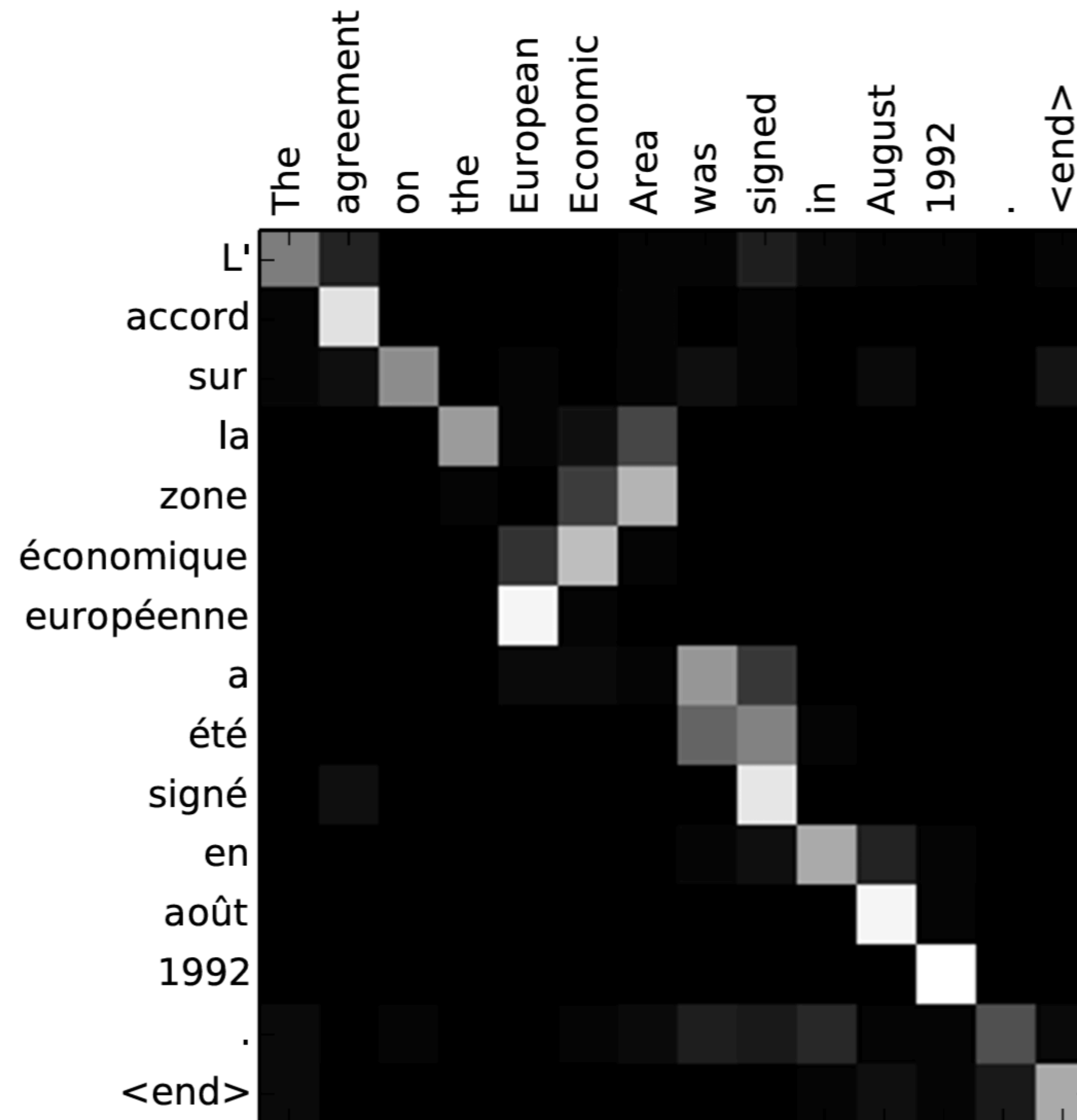
$[1, 0]$ $[1, 0]$ $[0, 1]$ $[1, 0]$
0 0 1 0

What will new attention values be with these keys?

Attention, Formally

- ▶ Original “dot product” attention: $s_i = k_i^T q$
- ▶ Scaled dot product attention: $s_i = k_i^T W q$
- ▶ Equivalent to having two weight matrices: $s_i = (W^K k_i)^T (W^Q q)$
- ▶ Other forms exist: Luong et al. (2015), Bahdanau et al. (2014) present some variants (originally for machine translation)

Attention Originally



Bahdanau, Cho, Bengio (2014)

Administrative details and recap

Neural Language Modeling

Attention

Self-attention

Transformers

Positional Encodings

Transformer Language Modeling

Self-attention

Contextualized Embeddings

RECAP

- ▶ Both RNNs and Transformers (and other models) can produce *contextualized embeddings*

$$\mathbf{e} = (e_1, e_2, \dots, e_n) \quad e_i = f(x_1, x_2, \dots, x_i)$$

$$\mathbf{x} = (x_1, x_2, \dots, x_n)$$

- ▶ unidirectional representation (only looks at past words)

$\mathbf{x} =$ *I visited New York. I had a really fun time going up the ____*

- ▶ Our current view of attention uses a single attention computation to compute what should go in ____
- ▶ Language modeling involves predicting each next token given the tokens that came before it...so we should do this for *all* positions

Self-Attention

Self-attention: every word is both a key and a query simultaneously

Q: seq len x d matrix (d = embedding dimension = 2 for these slides)

K: seq len x d matrix

$W^Q = \begin{pmatrix} 0 & 1 \\ 0 & 1 \end{pmatrix}$ no matter what the value is, we're going to look for Bs

$W^K = \begin{pmatrix} 10 & 0 \\ 0 & 10 \end{pmatrix}$ “booster” as before

Note: many ways to set up these weights that will be equivalent to this!

Self-Attention

$$E = \begin{pmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}$$

$$W^Q = \begin{pmatrix} 0 & 1 \\ 0 & 1 \end{pmatrix}$$

$$W^K = \begin{pmatrix} 10 & 0 \\ 0 & 10 \end{pmatrix}$$

$$Q = E (W^Q) = \begin{pmatrix} 0 & 1 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \end{pmatrix}$$

$$K = E (W^K) = \begin{pmatrix} 10 & 0 \\ 10 & 0 \\ 0 & 10 \\ 10 & 0 \end{pmatrix}$$

Scores $S = QK^T$ $S_{ij} = q_i \cdot k_j$

$\text{len} \times \text{len} = (\text{len} \times d) \times (d \times \text{len})$

Let's compute these now!

Self-Attention

$$E = \begin{pmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}$$

$$W^Q = \begin{pmatrix} 0 & 1 \\ 0 & 1 \end{pmatrix}$$

$$W^K = \begin{pmatrix} 10 & 0 \\ 0 & 10 \end{pmatrix}$$

$$Q = E (W^Q) = \begin{pmatrix} 0 & 1 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \end{pmatrix}$$

$$K = E (W^K) = \begin{pmatrix} 10 & 0 \\ 10 & 0 \\ 0 & 10 \\ 10 & 0 \end{pmatrix}$$

Scores $S = QK^T$ $S_{ij} = q_i \cdot k_j$

$\text{len} \times \text{len} = (\text{len} \times d) \times (d \times \text{len})$

Final step: softmax to get attentions A , then output is AE

*technically it's $A (EW^V)$, using a values matrix $V = EW^V$

Self-Attention (Vaswani et al.)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$Q = EW^Q, K = EW^K, V = EW^V$$

- ▶ Normalizing by $\sqrt{d_k}$ helps control the scale of the softmax, makes it less peaked
- ▶ This is just one head of self-attention — produce multiple heads via randomly initialize parameter matrices (more in a bit)

Self-Attention

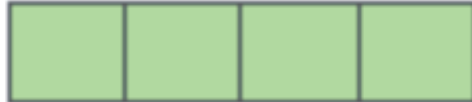
Alammar, *The Illustrated Transformer*

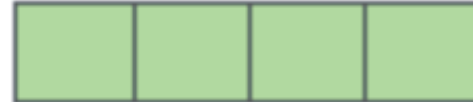
Input

Thinking

Machines

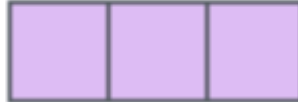
Embedding

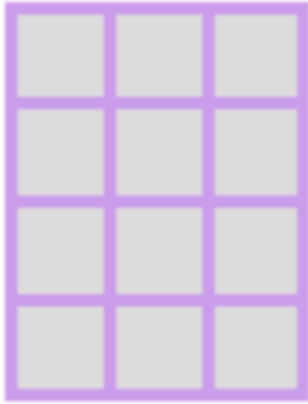
x_1 

x_2 

Queries

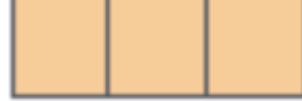
q_1 

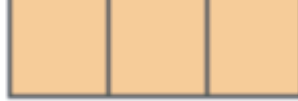
q_2 

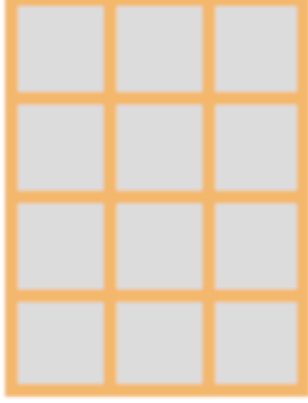


W^Q

Keys

k_1 

k_2 

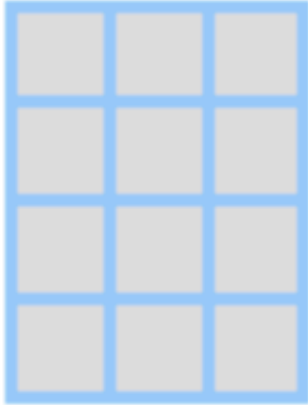


W^K

Values

v_1 

v_2 



W^V

Self-Attention

$$\begin{matrix} \text{X} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{W}^{\text{Q}} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} = \begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

$$\begin{matrix} \text{X} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{W}^{\text{K}} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} = \begin{matrix} \text{K} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

$$\begin{matrix} \text{X} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{W}^{\text{V}} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} = \begin{matrix} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

Alammar, *The Illustrated Transformer*

sent len x sent len (attn for each word to each other)

$$\text{softmax} \left(\frac{\begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{K}^{\text{T}} \\ \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \square & \square \\ \hline \end{array} \end{matrix}}{\sqrt{d_k}} \right) \begin{matrix} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

$$= \begin{matrix} \text{Z} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

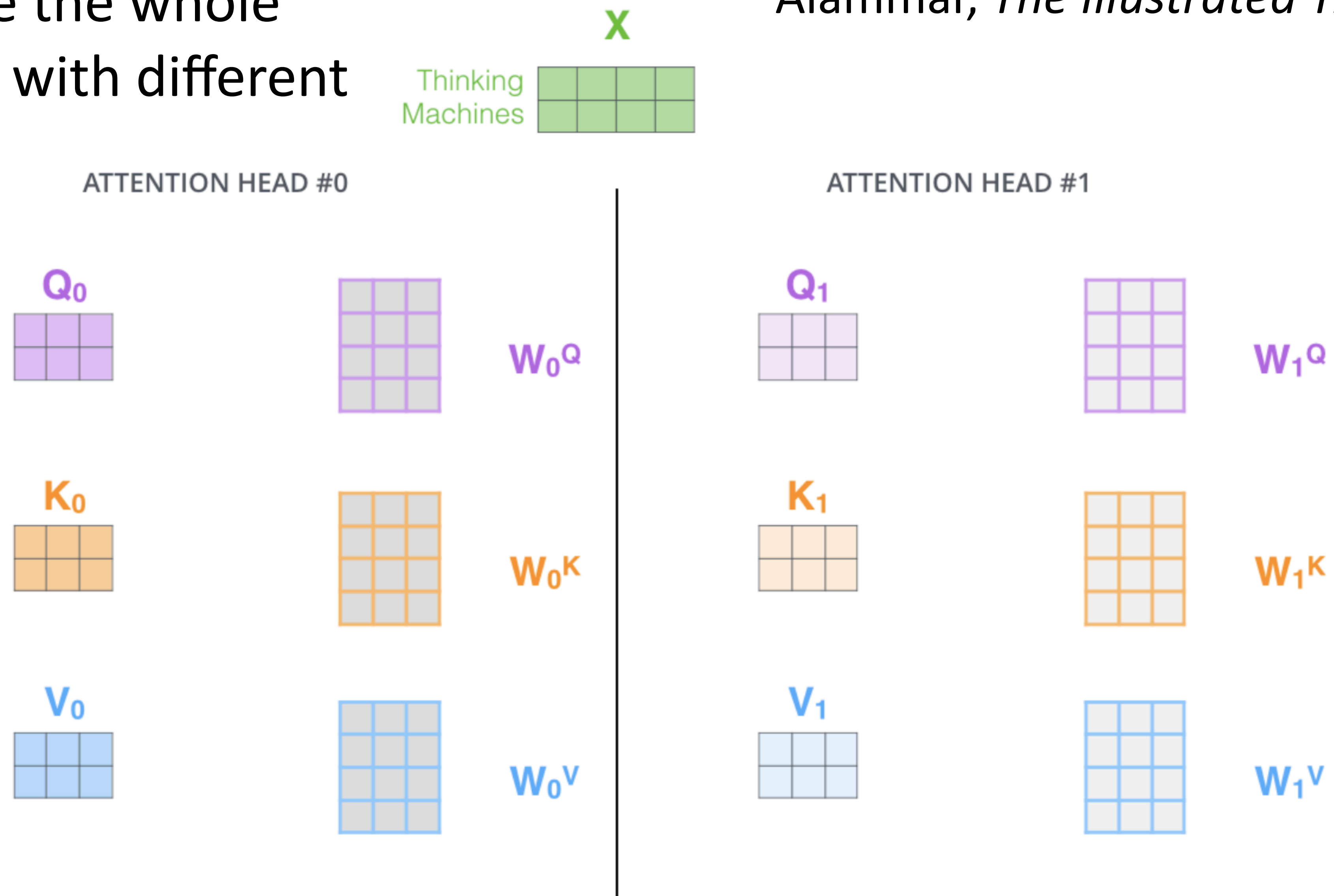
sent len x hidden dim

Z is a weighted combination of V rows

Multi-head Self-Attention

Just duplicate the whole computation with different weights:

Alammar, *The Illustrated Transformer*

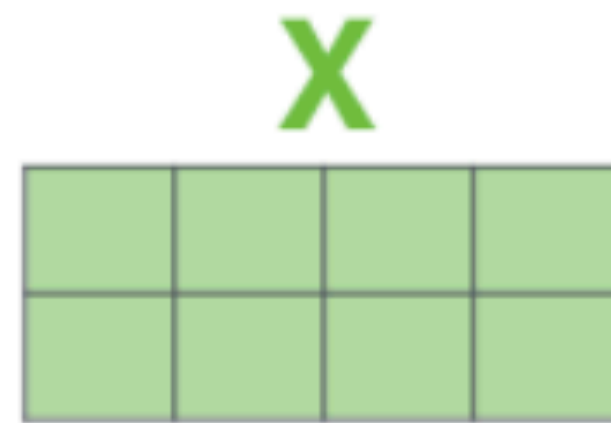


Multi-head Self-Attention

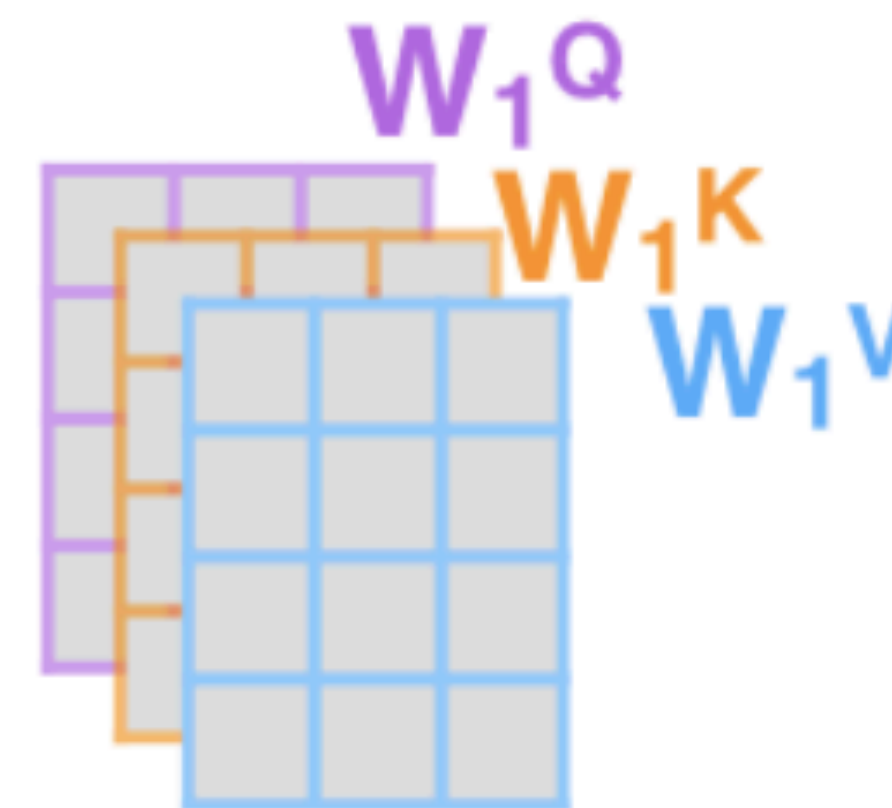
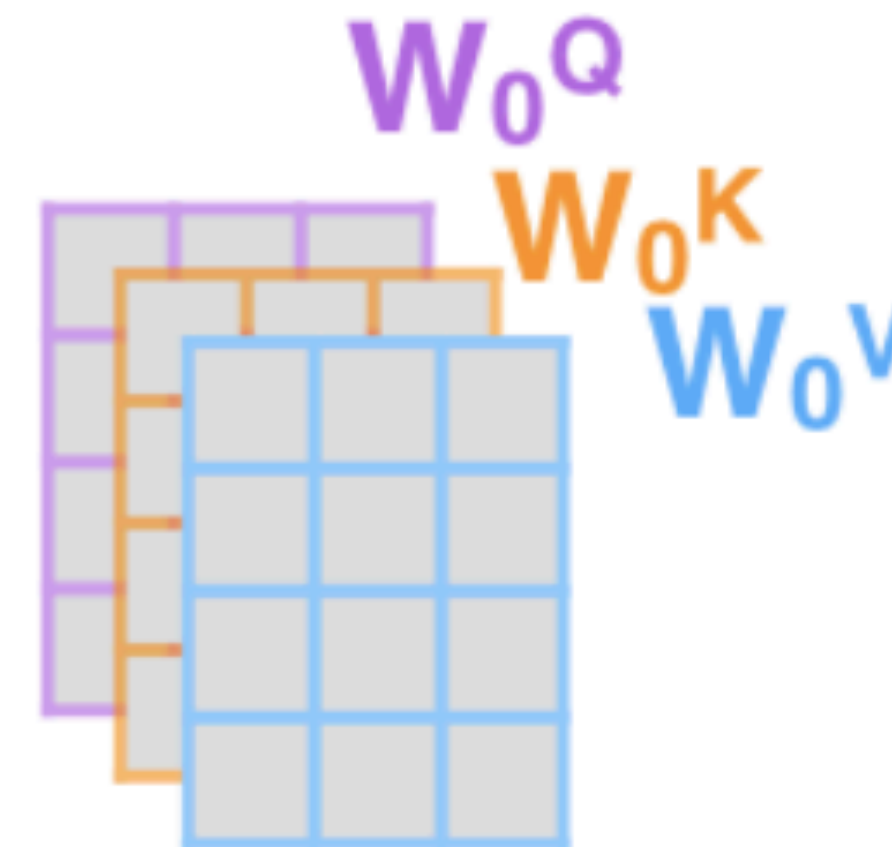
1) This is our input sentence*

Thinking
Machines

2) We embed each word*



3) Split into 8 heads.
We multiply **X** or **R** with weight matrices



Alammar, *The Illustrated Transformer*

* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

Multi-head Self-Attention

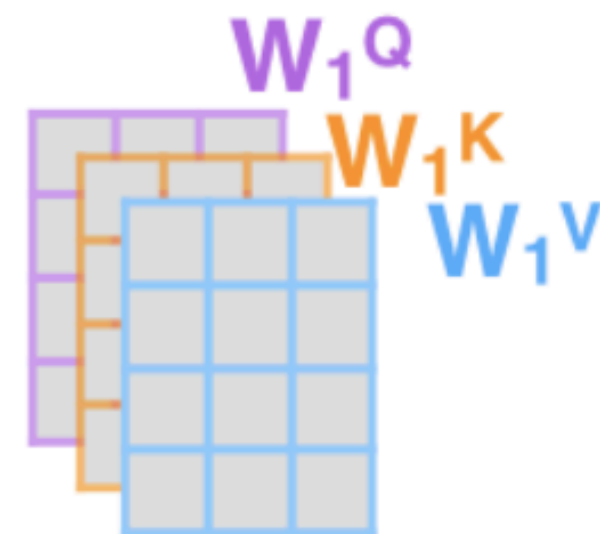
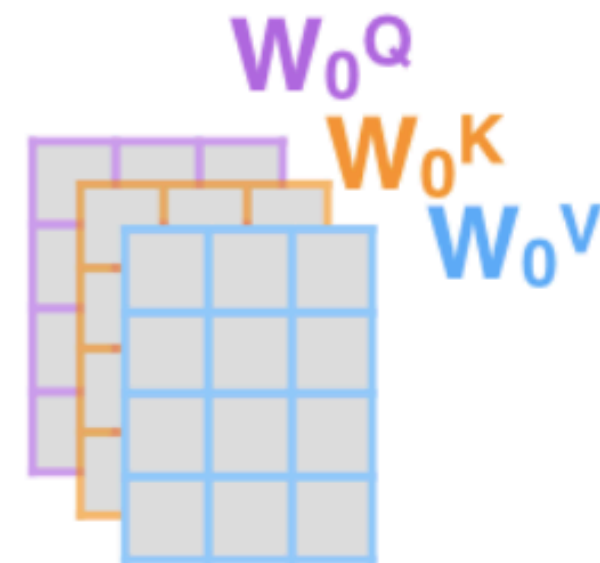
1) This is our input sentence*

Thinking
Machines

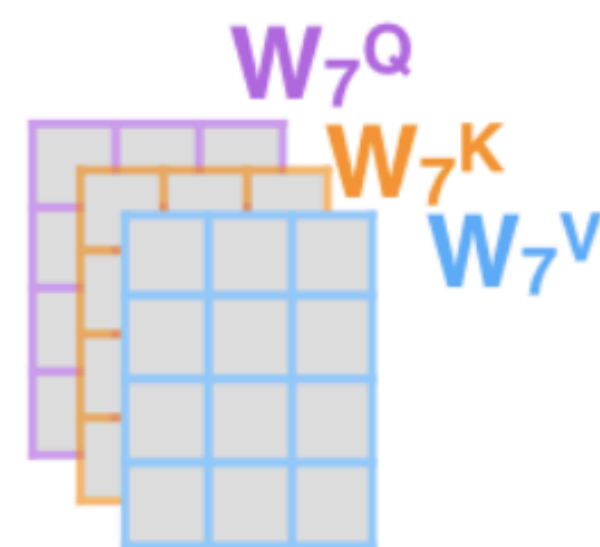
2) We embed each word*



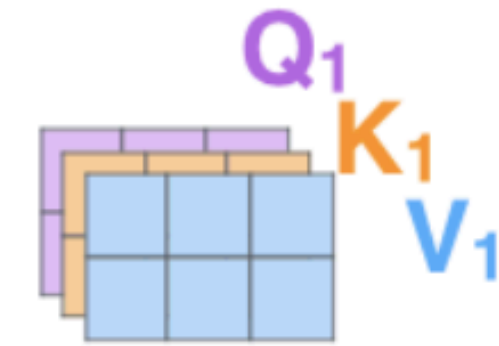
3) Split into 8 heads. We multiply X or R with weight matrices



...



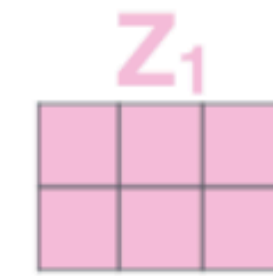
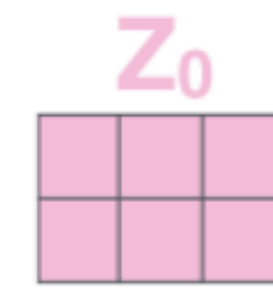
4) Calculate attention using the resulting $Q/K/V$ matrices



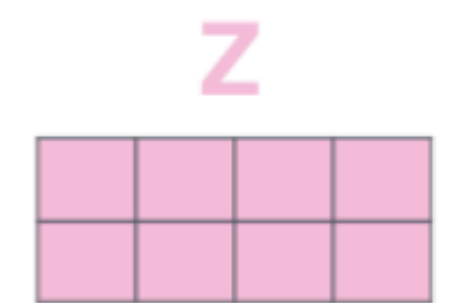
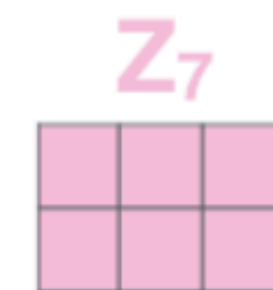
...



5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



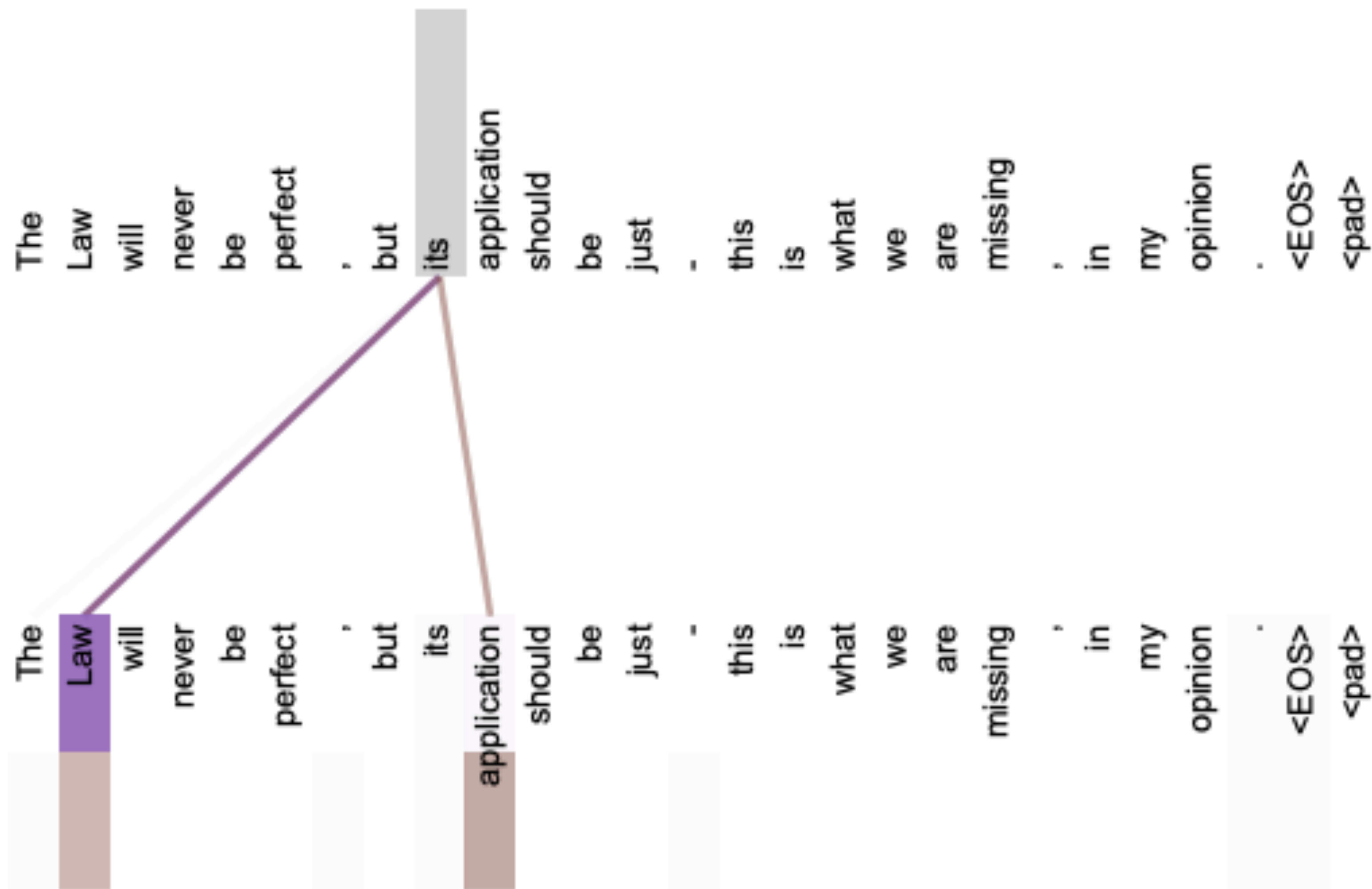
...



* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



Visualization & Intuition



- ▶ We might want to capture multiple places in the sentence at once, but softmax encourages picky distribution.
- ▶ You can think of each head capturing different dependencies: one for finding subject, one for finding object, etc...

Figure 4: Two attention heads, also in layer 5 of 6, apparently involved in anaphora resolution. Top: Full attentions for head 5. Bottom: Isolated attentions from just the word 'its' for attention heads 5 and 6. Note that the attentions are very sharp for this word.

Administrative details and recap

Neural Language Modeling

Attention

Self-attention

Transformers

Positional Encodings

Transformer Language Modeling

Transformers

Architecture

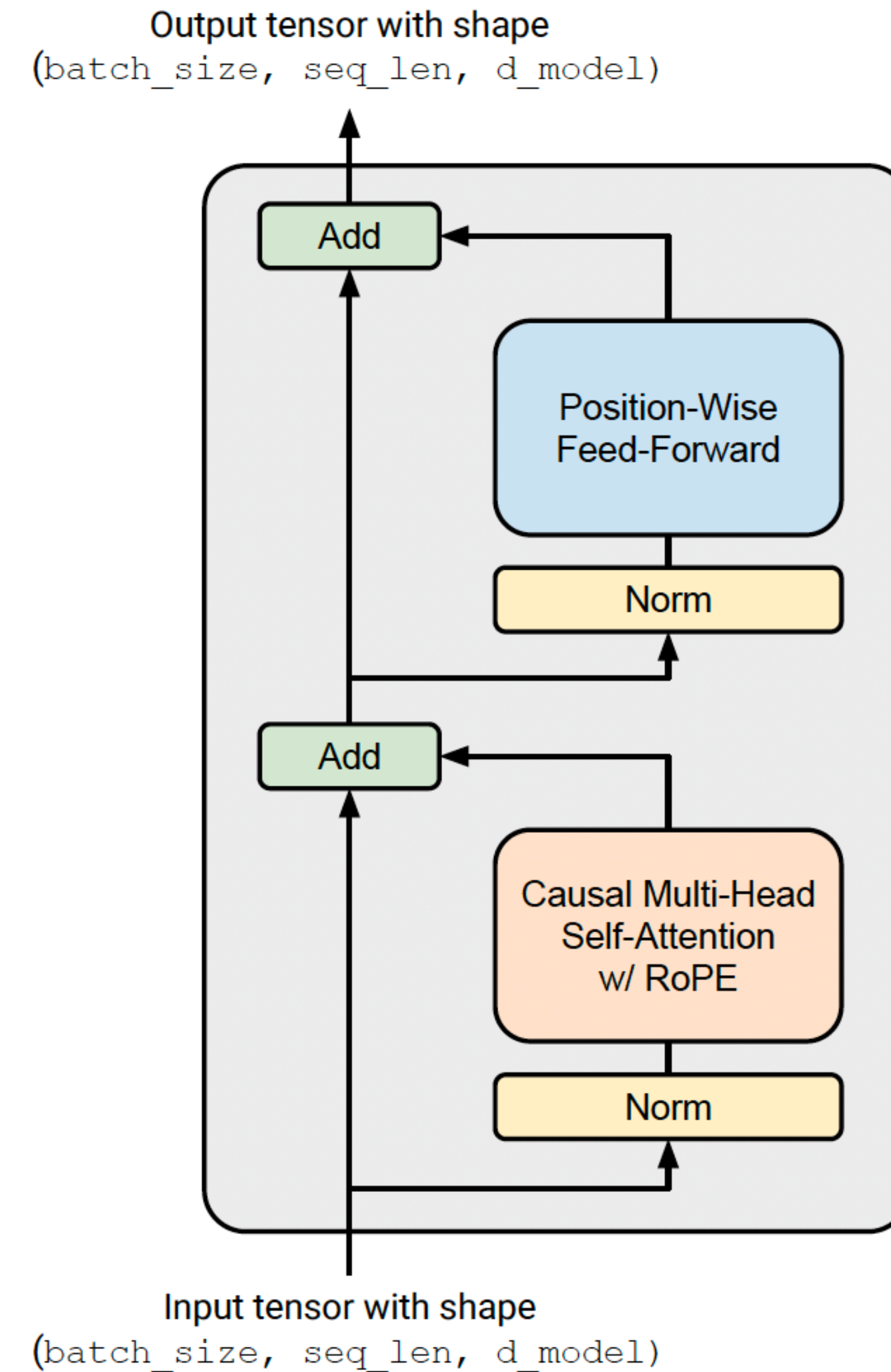
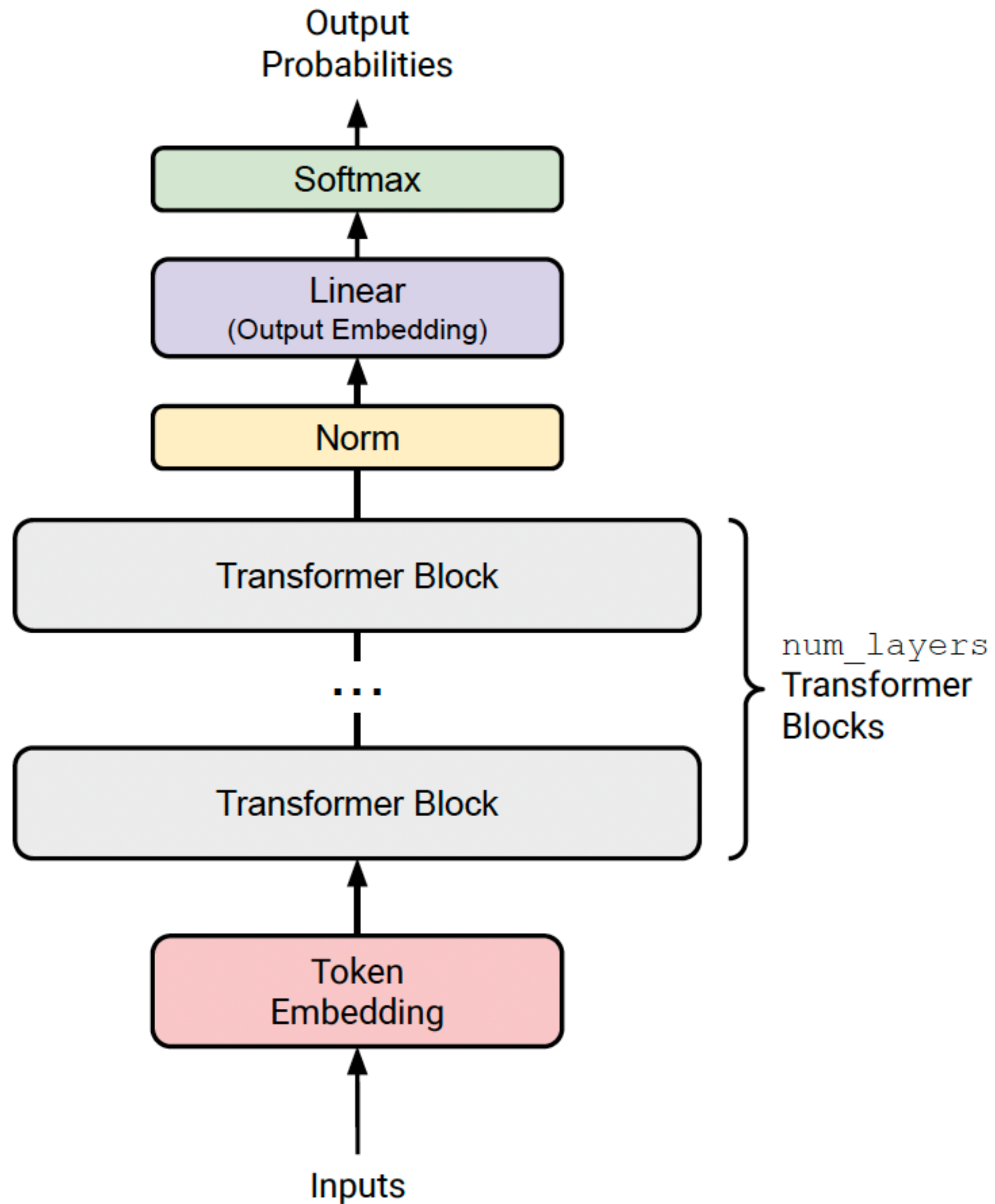
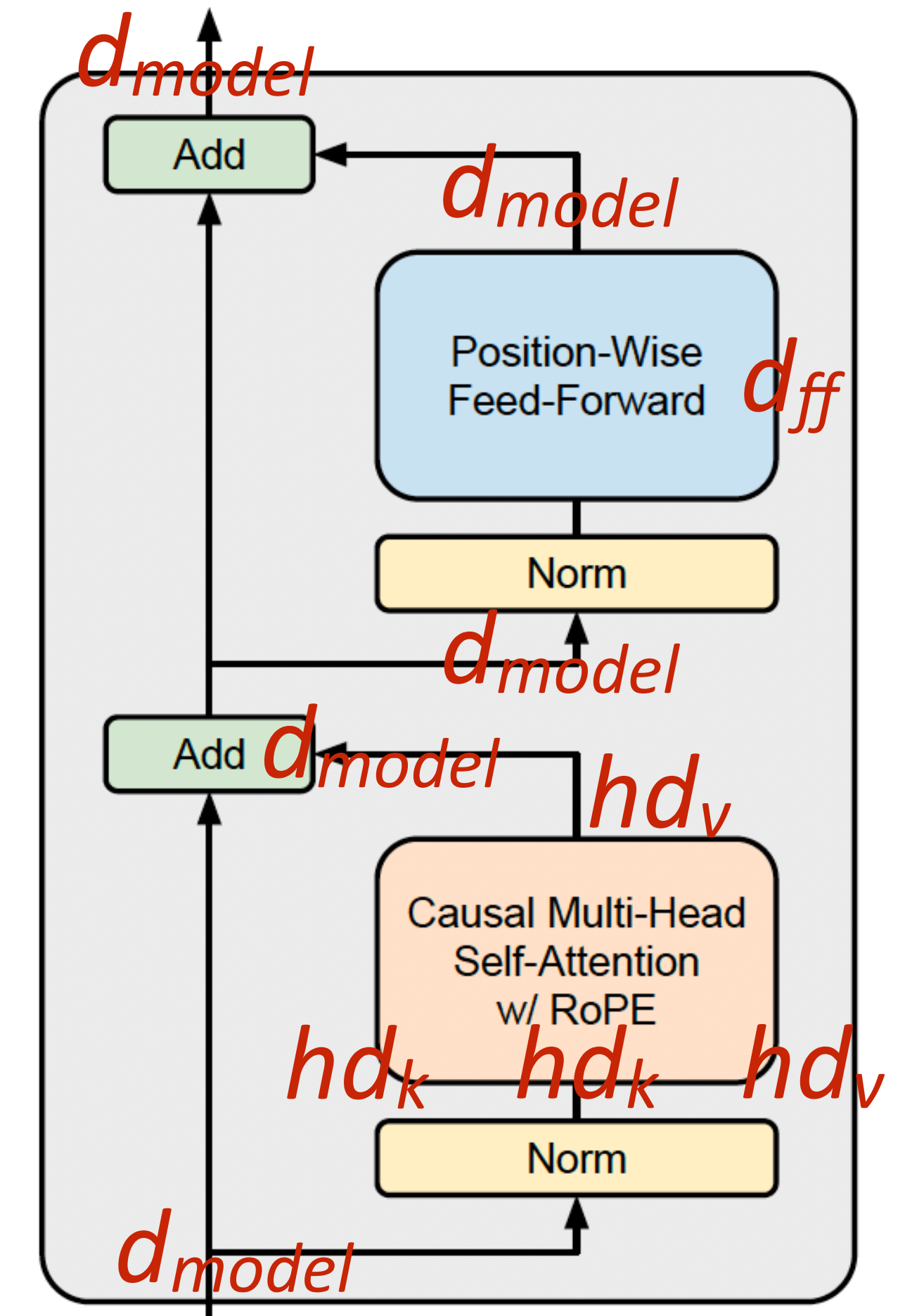


Figure credit:
Stanford CS336 A1

Dimensions

- ▶ Main vector size d_{model}
- ▶ Queries/keys: d_k , always smaller than d_{model} , often d_{model}/h (number of heads)
- ▶ Values: separate dimension d_v , output is multiplied by W^O which is $(d_v \times h) \times d_{model}$ so we can get back to d_{model}
- ▶ FFN can use a higher latent dimension

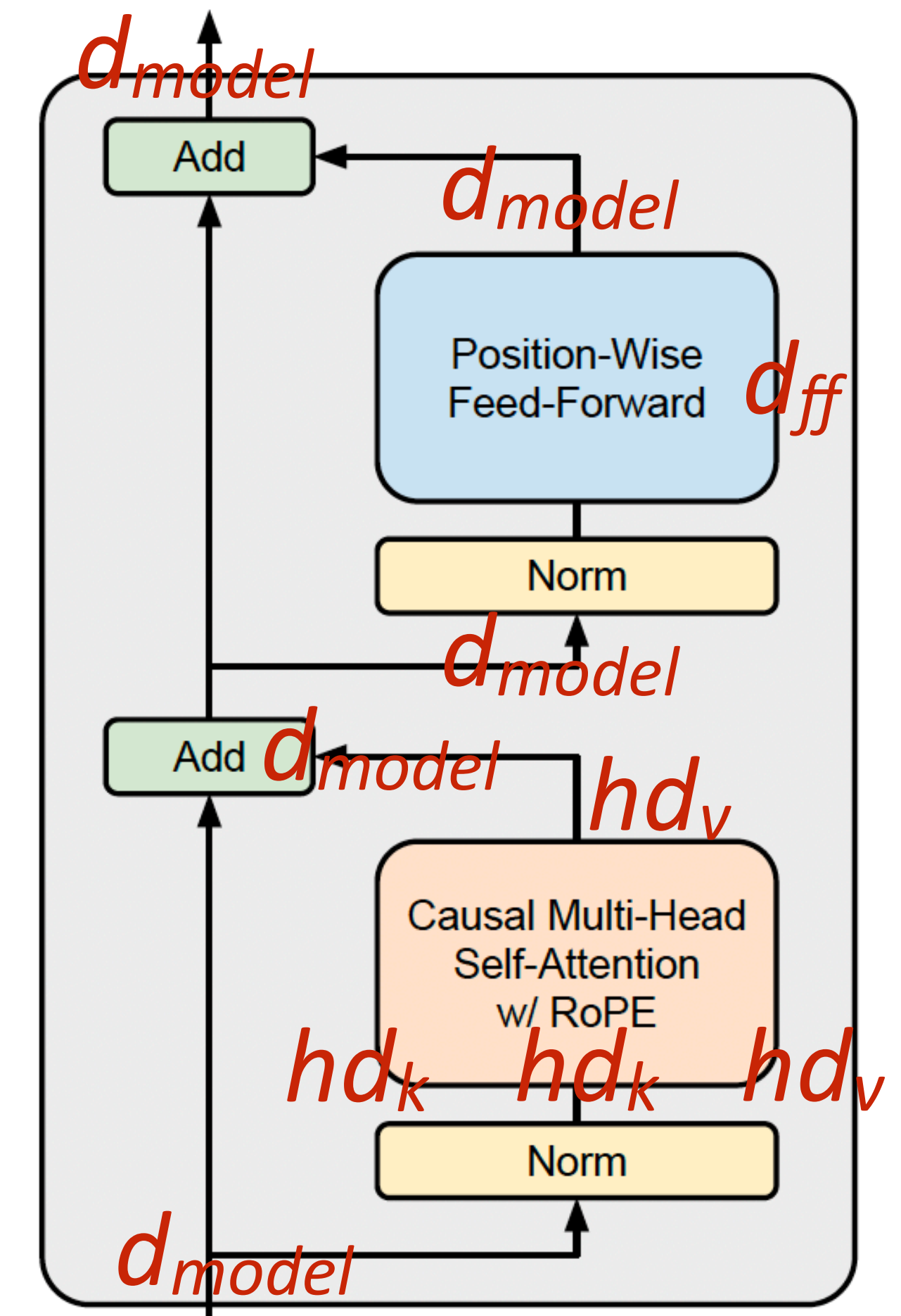


$$\text{FFN}(x) = \text{SwiGLU}(x, W_1, W_2, W_3) = W_2(\text{SiLU}(W_1 x) \odot W_3 x)$$

Transformer Architecture

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}
GPT-3 Small	125M	12	768	12	64
GPT-3 Medium	350M	24	1024	16	64
GPT-3 Large	760M	24	1536	16	96
GPT-3 XL	1.3B	24	2048	24	128
GPT-3 2.7B	2.7B	32	2560	32	80
GPT-3 6.7B	6.7B	32	4096	32	128
GPT-3 13B	13.0B	40	5140	40	128
GPT-3 175B or “GPT-3”	175.0B	96	12288	96	128

From GPT-3; d_{head} is our d_k



FLOPs

1	description	FLOPs / update	% FLOPS MHA	% FLOPS FFN	% FLOPS attn	% FLOPS logit
8	OPT setups					
9	760M	4.3E+15	35%	44%	14.8%	5.8%
10	1.3B	1.3E+16	32%	51%	12.7%	5.0%
11	2.7B	2.5E+16	29%	56%	11.2%	3.3%
12	6.7B	1.1E+17	24%	65%	8.1%	2.4%
13	13B	4.1E+17	22%	69%	6.9%	1.6%
14	30B	9.0E+17	20%	74%	5.3%	1.0%
15	66B	9.5E+17	18%	77%	4.3%	0.6%
16	175B	2.4E+18	17%	80%	3.3%	0.3%

Administrative details and recap

Neural Language Modeling

Attention

Self-attention

Transformers

Positional Encodings

Transformer Language Modeling

Positional Encodings

Running Example

AAAAAA**A**

- ▶ All As = last letter is A; any B = last letter is B

ABAAAA**B**

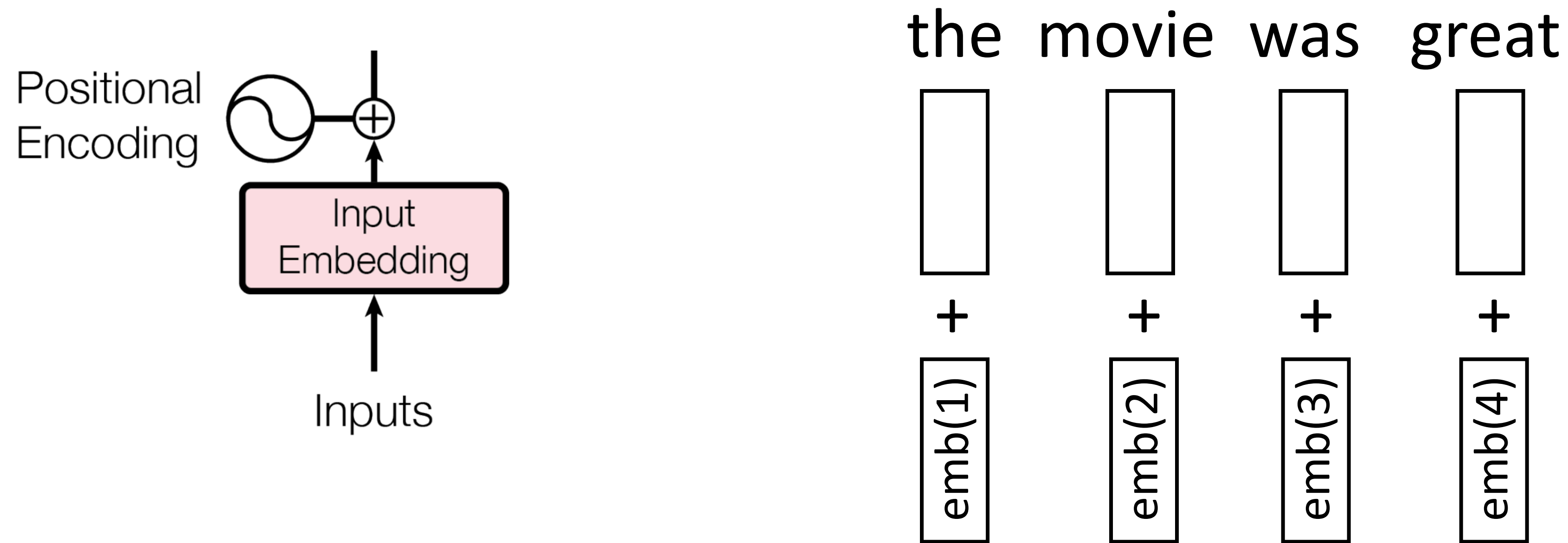
ABAABA**B**

AAAABA**B**

BAAAAAAAAAAAAAAAAAAAAAAAAAA**B**

- ▶ This didn't rely on self-attention being able to distinguish different positions.
- ▶ If we want to detect "B followed by 3 As", our approach so far may* not work well. (*we'll come back to this)

Absolute Position Encodings (BERT, etc.)

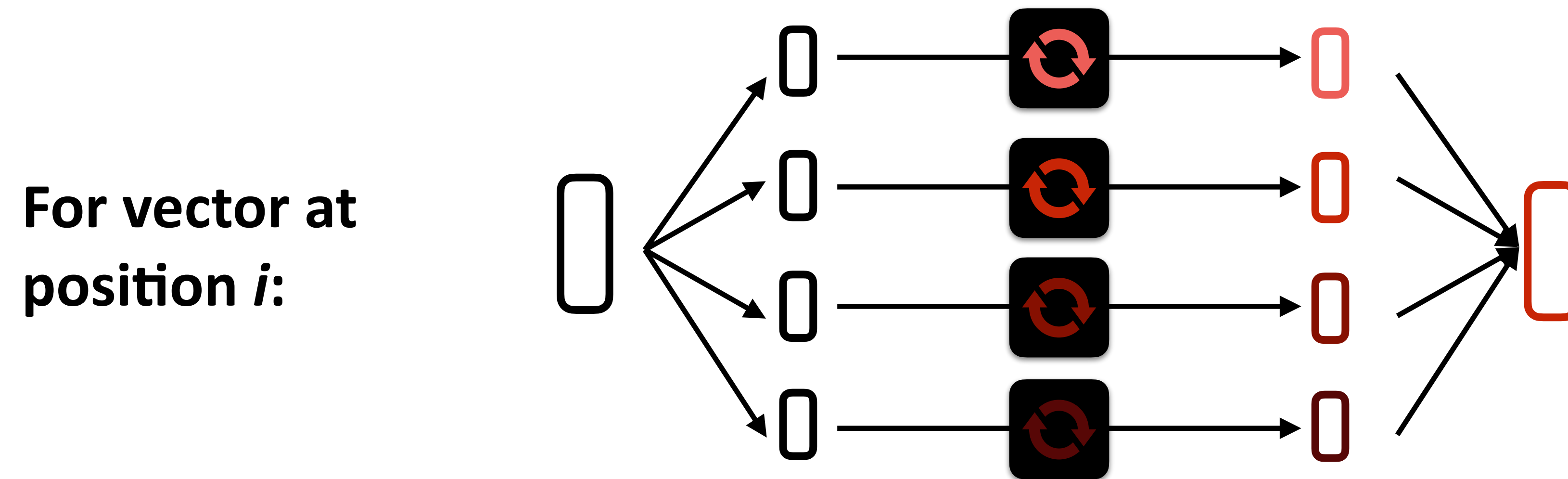


- ▶ Encode each sequence position as an integer, add it to the word embedding vector
- ▶ Why does this work?
- ▶ RoPE is the positional encoding scheme actually used

RoPE (Jianlin Su et al., 2021)



Goal: encode positional information in each vector.

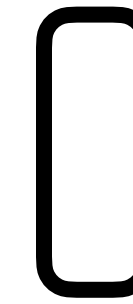
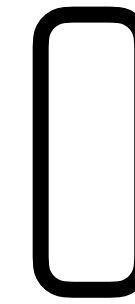
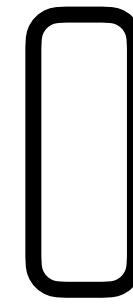


Step 1: Break into $d/2$ vectors in $\mathbb{R}^2$

Step 2: Rotate each one by an amount depending on i and the vector index

RoPE (Jianlin Su et al., 2021)

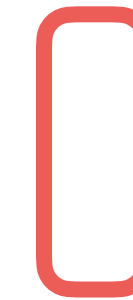
d -dim
vectors



increasing token position i (going through sentence)



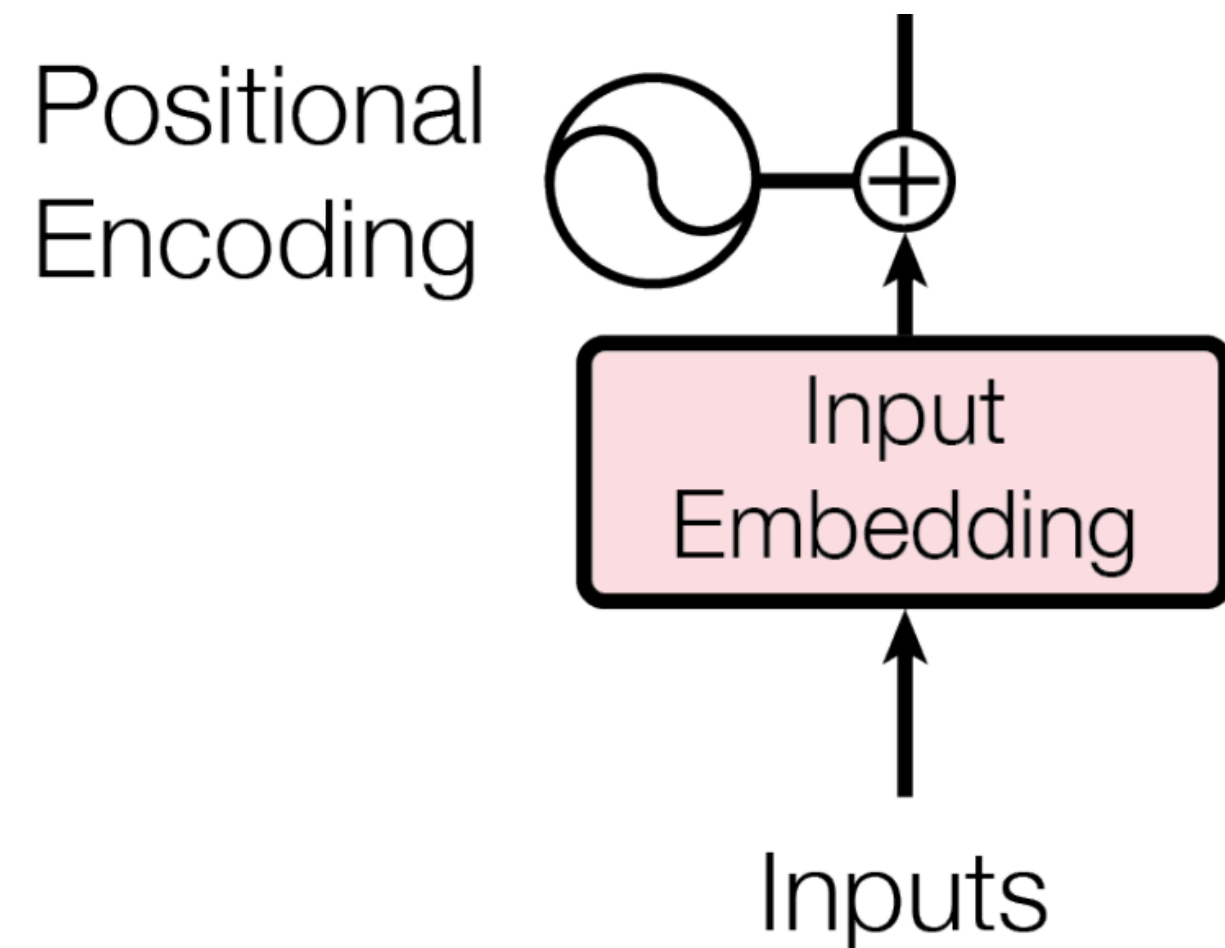
RoPE layer



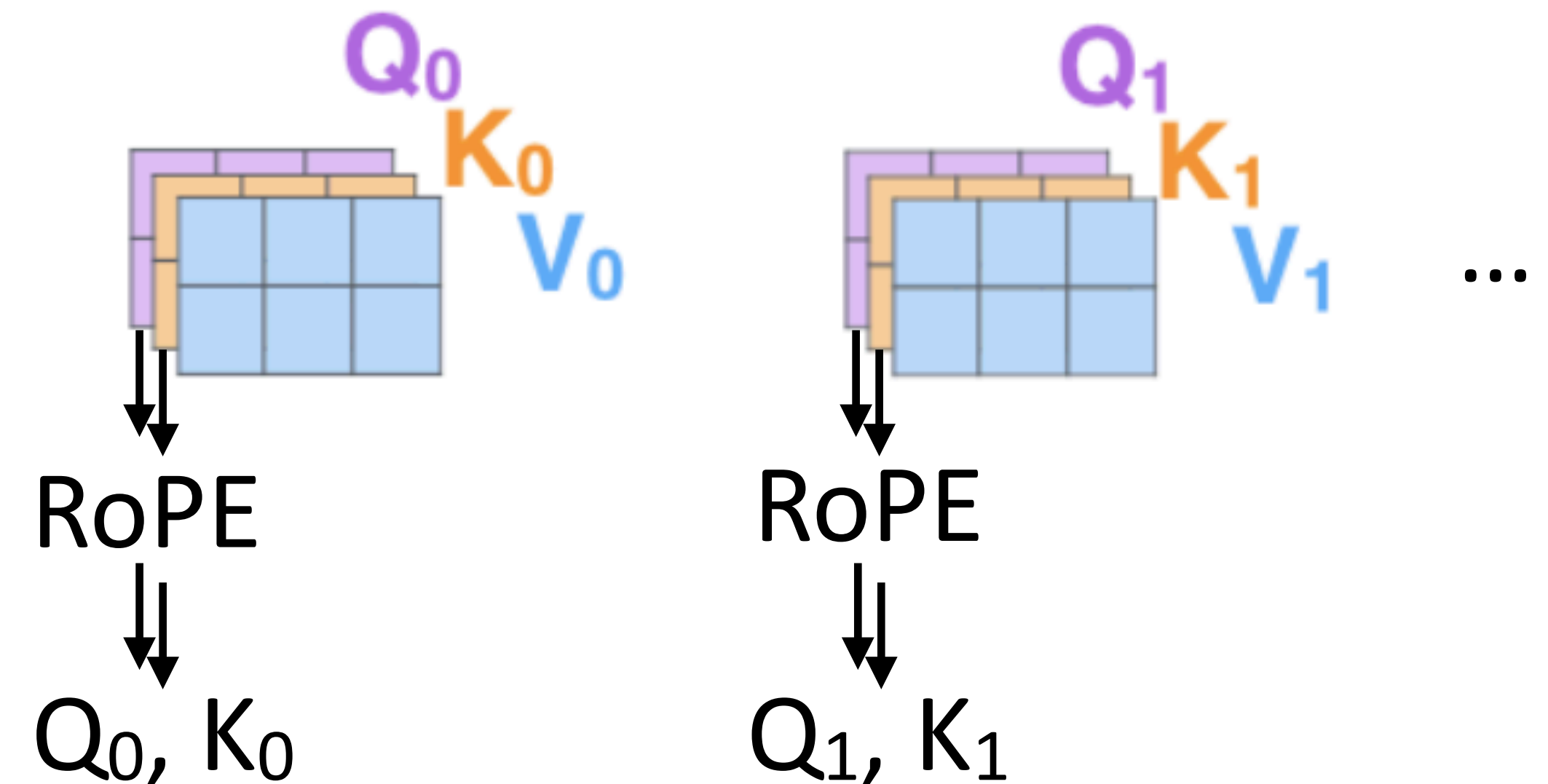
increasing rotation amount

How does this help encode position for a Transformer?

Where are PEs used?



Classical Vaswani et al.
Transformer (2017): added to
input



Modern practice: Apply RoPE to
Qs and Ks right before self-
attention

- How do these methods differ?

Administrative details and recap

Neural Language Modeling

Attention

Self-attention

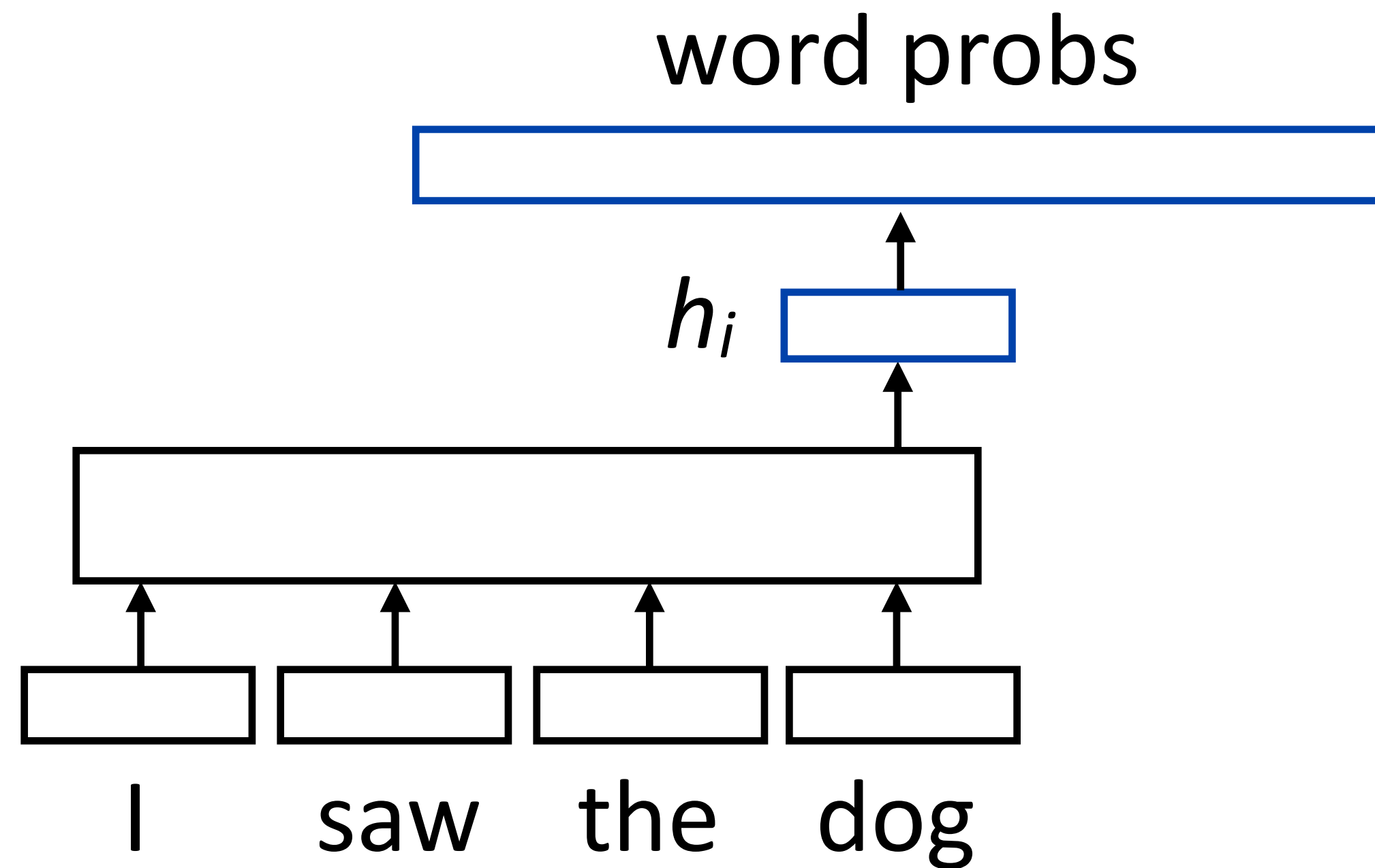
Transformers

Positional Encodings

Transformer Language Modeling

Transformer Language Modeling

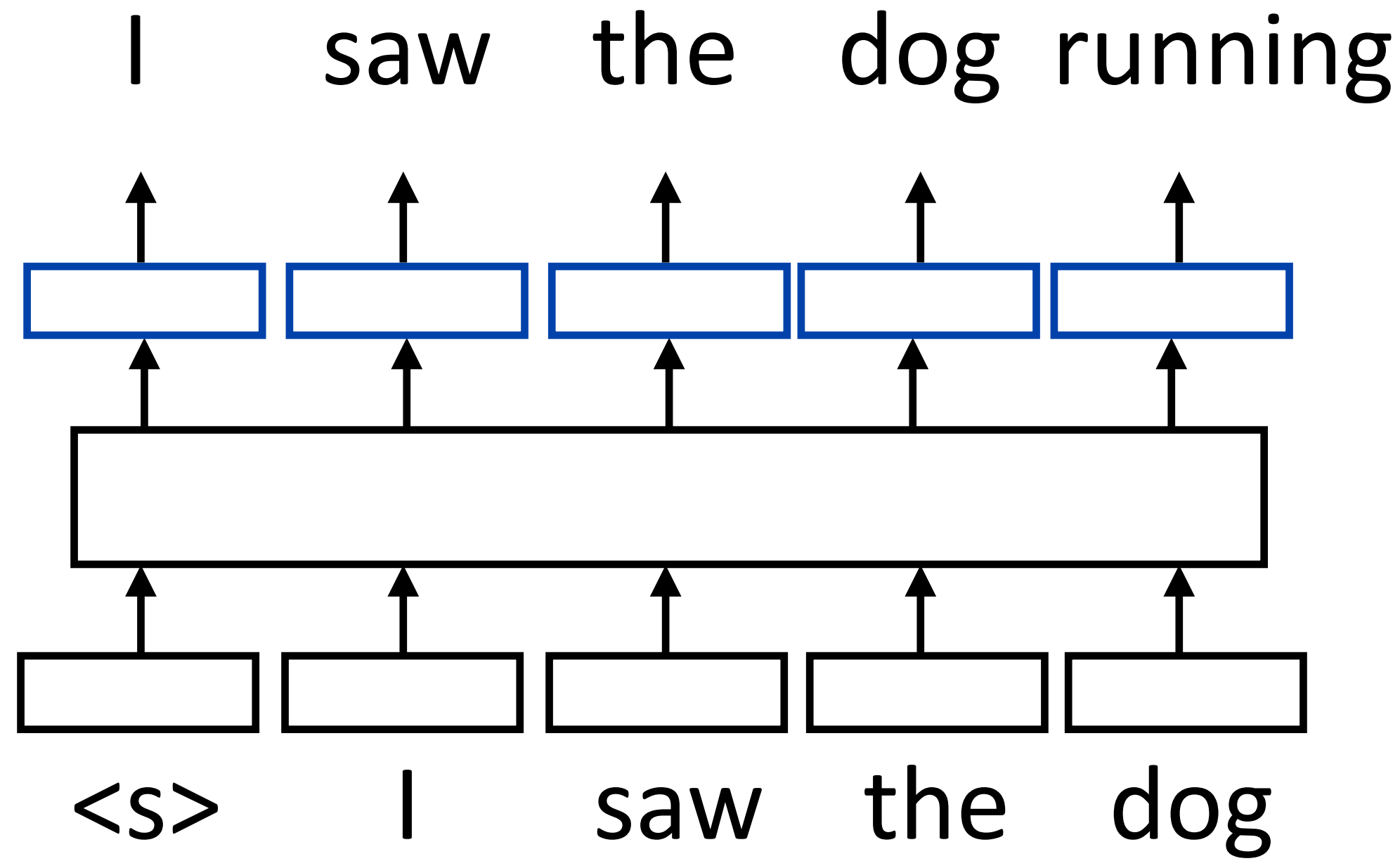
Transformer Language Modeling



$$P(w|\text{context}) = \text{softmax}(W\mathbf{h}_i)$$

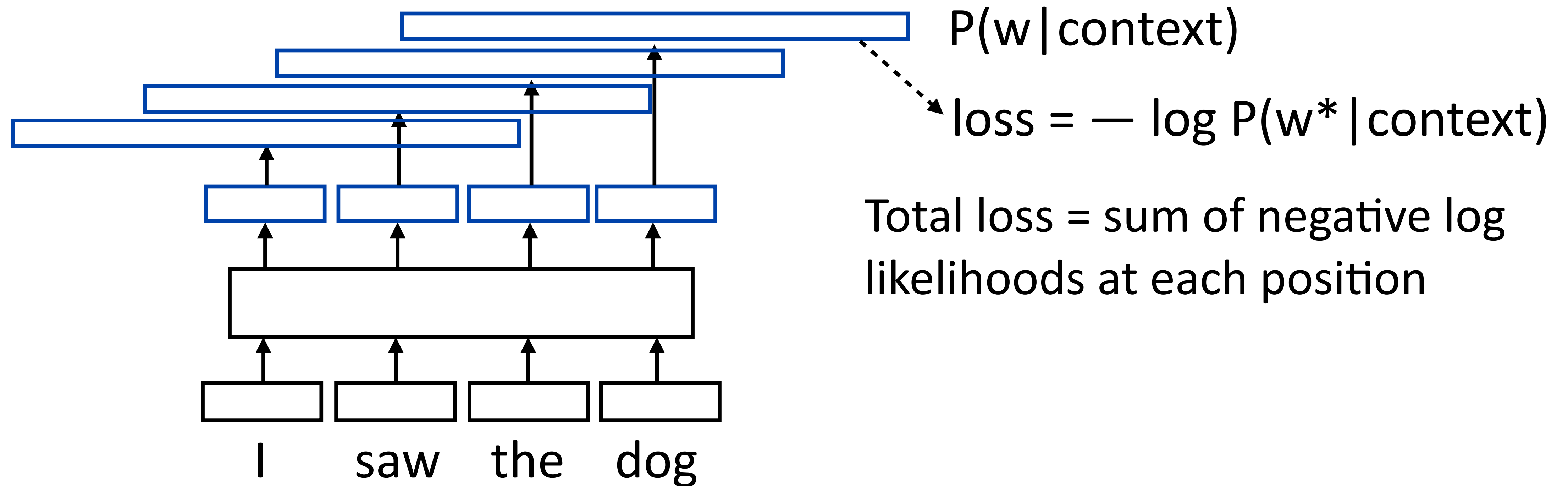
W is a (vocab size) x (hidden size) matrix

Training Transformer LMs



- ▶ Input is a sequence of words, output is those words shifted by one,
- ▶ Allows us to train on predictions across several timesteps simultaneously (similar to batching but this is NOT what we refer to as batching)

Training Transformer LMs

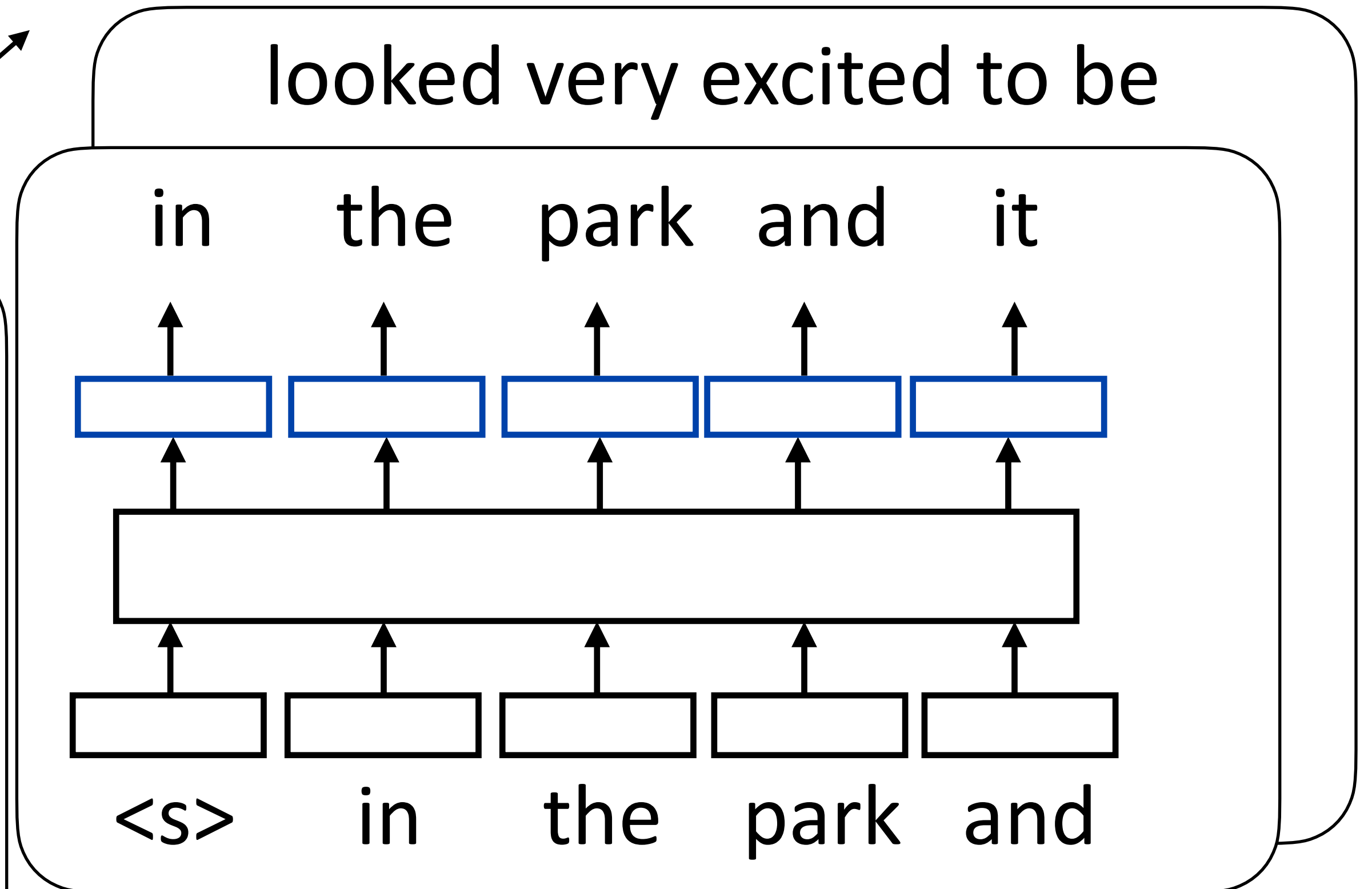
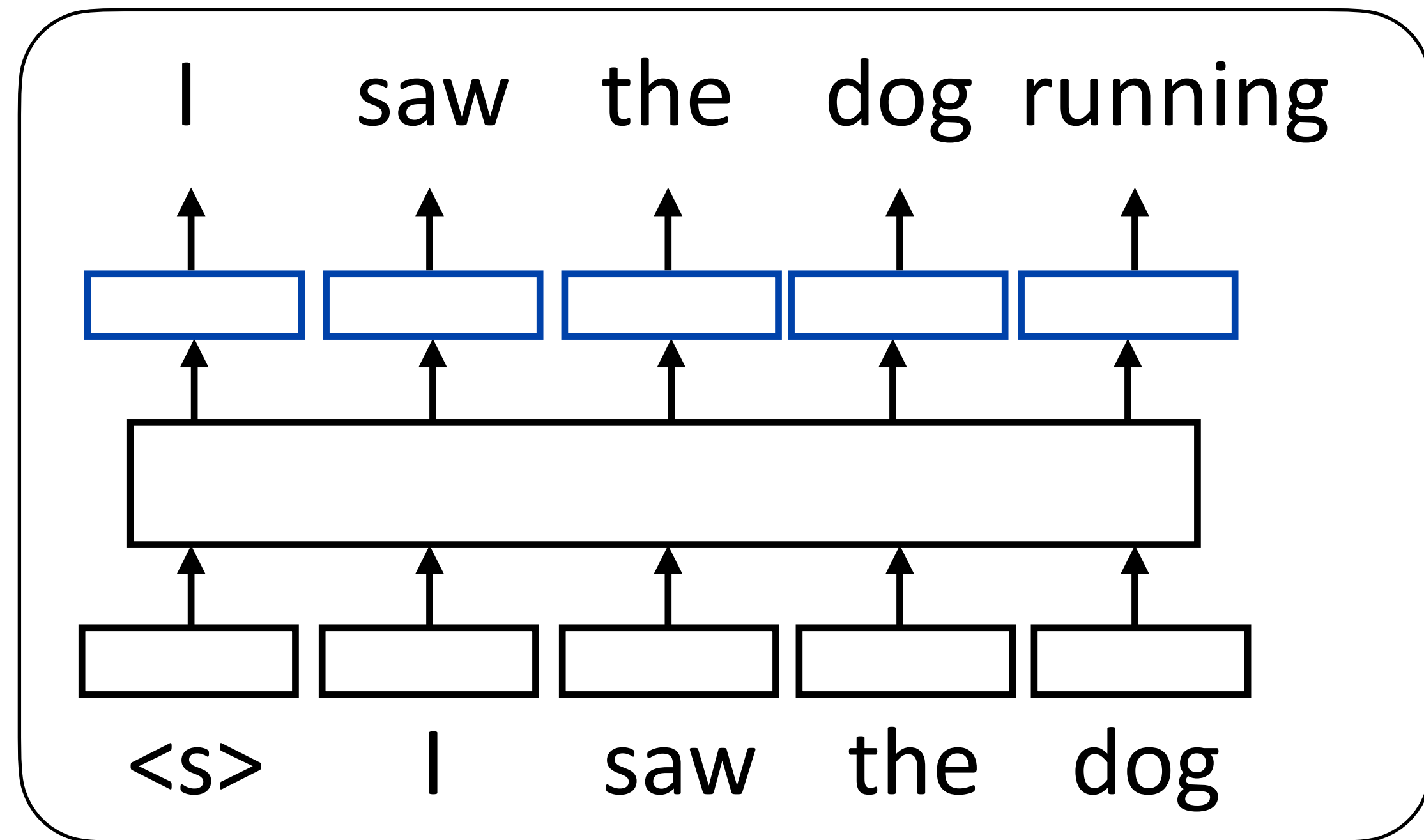


- ▶ Parallel inference across several tokens at training time, but at decoding time, tokens are generated one at a time

Batched LM Training

I saw the dog running in the park and it looked very excited to be there

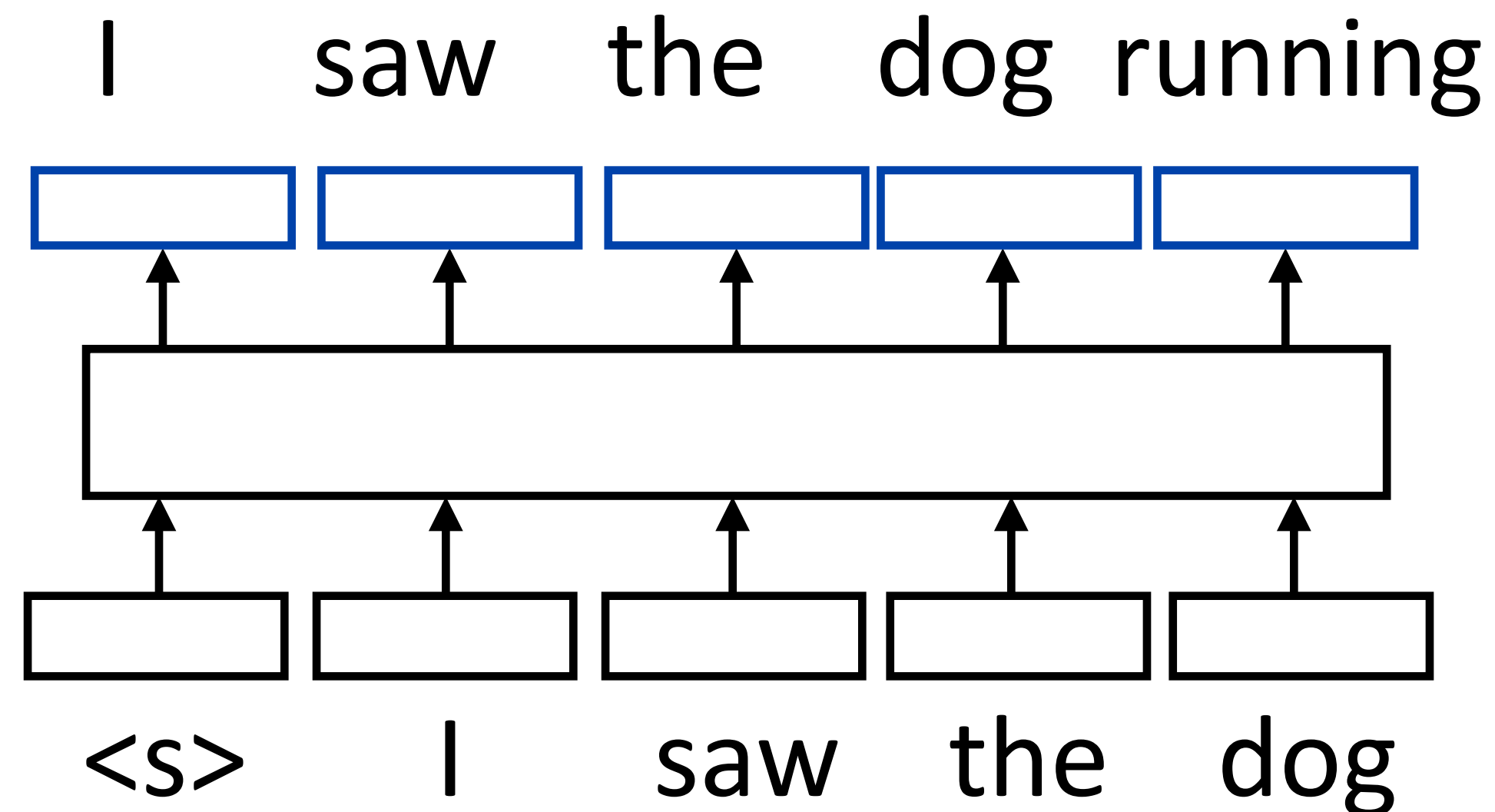
batch dim



- ▶ Multiple sequences **and** multiple timesteps per sequence

A Small Problem with Transformer LMs

- ▶ This Transformer LM as we've described it will *easily* achieve perfect accuracy. Why?



- ▶ With standard self-attention: “I” attends to “saw” and the model is “cheating”. How do we ensure that this doesn’t happen?

Attention Masking



- ▶ We want to mask out everything in red (an upper triangular matrix)

Administrative details and recap

Neural Language Modeling

Attention

Self-attention

Transformers

Positional Encodings

Transformer Language Modeling